

THE RANDOM PRIORITY MECHANISM IS UNIQUELY SIMPLE, EFFICIENT, AND FAIR

MAREK PYCIA

Department of Economics, University of Zurich

PETER TROYAN

Department of Economics, University of Virginia

Random Priority is a popular mechanism used to allocate a set of objects to a set of agents without the use of monetary transfers. Random Priority is appealing because it satisfies desirable efficiency, fairness, and incentive properties. Is it the only mechanism with these properties? We answer this long-standing question in the positive: Random Priority is the unique mechanism that is Pareto efficient, symmetric, and obviously strategy-proof.

KEYWORDS: Random Priority, obvious strategy-proofness, simplicity, fairness, symmetry, Pareto efficiency, object allocation, extensive-form games.

1. INTRODUCTION

CONSIDER THE PROBLEM of allocating n indivisible objects to n agents without the use of monetary transfers.¹ Examples of such problems include assigning school seats to K12 students, dormitory rooms to college students, tasks to workers, offices to professors, or time slots on a common machine. A classic and oft-used solution to this problem is the *Random Priority* mechanism in which the random allocation is created by drawing an or-

Marek Pycia: marek.pycia@econ.uzh.ch

Peter Troyan: troyan@virginia.edu

This paper subsumes the analysis of Random Priority from 2016–2022 drafts of Pycia and Troyan (2023b) (initially presented and distributed under the title “Obvious Dominance and Random Priority”). For their comments, we would like to thank Itai Ashlagi, Sarah Auster, Eduardo Azevedo, Roland Benabou, Dirk Bergemann, Tilman Börgers, Ernst Fehr, Dino Gerardi, Ben Golub, Yannai Gonczarowski, Ed Green, Samuel Haefner, Rustamdjan Hakimov, Stine Helmke, Shaowei Ke, Fuhito Kojima, Simon Lazarus, Jiangtao Li, Shengwu Li, Giorgio Martini, Nelson Mesker, Stephen Morris, Nick Netzer, Ryan Oprea, Ran Shorrer, Erling Skancke, Utku Ünver, Martin Valdevenito, Roberto Weber, anonymous referees, the Eco 514 students at Princeton, and the audiences at the 2016 NBER Market Design workshop, NEG-T16, NC State, ITAM, NSF/CEME Decentralization, the Econometric Society Meetings, UBC, the Workshop on Game Theory at NUS, UVa, ASSA, MATCH-UP, EC’19 (the Best Paper prize), ESSET, Wash U, Maryland, Warsaw Economic Seminars, ISI Delhi, Notre Dame, UCSD, Columbia, Rochester, Brown, Glasgow, Singapore Management University, Matching in Practice, Essex, European Meeting on Game Theory, GMU, Richmond Fed, Israel Theory Seminar, USC, Collegio Carlo Alberto, BC, Penn State, the 2024 Conference on Mechanism and Institution Design, and the 2025 Workshop on Normative Aspects of Economic Policy. Pycia gratefully acknowledges the support of the UCLA Department of Economics and the William S. Dietrich II Economic Theory Center at Princeton. Troyan gratefully acknowledges support from the Bankard Fund for Political Economy and the Roger Sherman Fellowship at the University of Virginia.



This project has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program grant agreement No 866376.

¹In the main body of the paper, we focus on the canonical environment with equal numbers of agents and objects. In Appendix D, we show that our results remain valid with no restrictions on the relative cardinalities of these sets.

© 2026 The Authors. *Econometrica* published by Wiley Periodicals LLC on behalf of The Econometric Society. Marek Pycia is the corresponding author on this paper. This is an open access article under the terms of the [Creative Commons Attribution](#) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

dering of agents uniformly at random and calling the agents, one-by-one, to select their favorite object from those that were not selected by earlier agents.²

The popularity of Random Priority largely derives from its desirable efficiency, fairness, and incentive (or simplicity) properties. A long-standing open question has been whether any *other* mechanism also satisfies such properties, or whether Random Priority is the unique mechanism to do so. On the positive side, when $n = 3$, [Bogomolnaia and Moulin \(2001\)](#) characterize Random Priority as the unique mechanism satisfying Pareto efficiency, strategy-proofness, and equal treatment of equals; [Brandt, Greger, and Romen \(2023\)](#) extended this result to $n = 4$.³ However, [Basteck and Ehlers \(2024\)](#) show it does not extend further: for $n \geq 5$, they provide an example of a mechanism that is strategy-proof, Pareto efficient, satisfies equal treatment of equals, and not only is different than Random Priority but also leads to distributions of individual agents' outcomes that are different than in Random Priority.⁴

A characterization of Random Priority thus requires strengthening at least one of strategy-proofness, Pareto efficiency, or equal treatment of equals.⁵ Random Priority already fails natural strengthenings of Pareto efficiency: beyond very small markets, it is neither ordinally efficient in the sense of [Bogomolnaia and Moulin \(2001\)](#) nor ex ante efficient in the sense of [Zhou \(1990\)](#).⁶ Random Priority does, however, satisfy natural strengthenings of the equal-treatment property and of strategy-proofness: it is *symmetric* and its extensive-form implementation is *obviously strategy-proof (OSP)*.⁷ Symmetry, which is also called anonymity, is a natural fairness requirement that says that the outcome of a mechanism should not depend on the identities of the agents. Obvious strategy-proofness is a strengthening of strategy-proofness introduced by [Li \(2017\)](#), who establishes its desirability as a simplicity criterion for designers who want to ensure that a mechanism can be recognized as strategy-proof by agents unable to engage in contingent reasoning. More so, the possibility of obviously strategy-proof extensive-form implementation matters even for designers restricted to static mechanisms, as it allows such designers to explain the mechanism in a simple way.⁸

²Random Priority is also called Random Serial Dictatorship; see, for example, [Abdulkadiroğlu and Sönmez \(1998\)](#).

³The result for $n = 4$ was also proven by Ehlers and Ünver (private communication, 2010) and Echenique, Pomatto, Root, Sandomirskiy, and Tamuz (private communication, 2020). [Pycia and Ünver \(2015\)](#) discuss methodological tools developed in a failed attempt to prove the conjecture.

⁴Prior to [Basteck and Ehlers \(2024\)](#), [Erdil \(2014\)](#) shows that the classic axioms of Pareto efficiency, equal treatment, and strategy-proofness do not characterize Random Priority when the number of agents and objects are different, and [Pycia and Troyan \(2023a\)](#) construct mechanisms different from Random Priority that are strongly obviously strategy-proof (and hence obviously strategy-proof and strategy-proof), Pareto efficient, and satisfy equal treatments of equals.

⁵We focus on finite markets of any fixed size. In large markets with many copies of each object, [Liu and Pycia \(2011\)](#) show that asymptotically all ordinally efficient, equal treatment, strategy-proof mechanisms with small agents have the same marginal distributions as Random Priority (Random Priority satisfies ordinal efficiency asymptotically ([Che and Kojima \(2010\)](#)) and, in large markets, ex post and ordinal efficiency coincide ([Liu and Pycia \(2011\)](#))).

⁶[Bogomolnaia and Moulin \(2001\)](#) show that Pareto efficiency is equivalent to ordinal efficiency for $n = 3$ but no ordinally efficient and strategy-proof mechanism satisfies equal treatment for $n \geq 4$. For $n \geq 3$, [Zhou \(1990\)](#) shows that there is no strategy-proof and ex ante efficient mechanism that satisfies equal treatments of equals. Random Priority fails ex ante efficiency even asymptotically, as can be inferred from [Abdulkadiroğlu, Che, and Yasuda \(2011\)](#), [Featherstone and Niederle \(2016\)](#), and [Miralles \(2008\)](#).

⁷Random Priority fails envy-freeness (another natural strengthening of equal treatment) as shown by [Bogomolnaia and Moulin \(2001\)](#).

⁸For the importance of simple descriptions, see, for example, [Bó and Hakimov \(2023\)](#), [Breitmoser and Schweighofer-Kodritsch \(2019\)](#), and [Gonczarowski, Heffetz, and Thomas \(2023\)](#). An experimental literature—

Can these properties be used to obtain a characterization of Random Priority? That is, is Random Priority the only mechanism that is Pareto efficient, symmetric, and obviously strategy-proof? The main result of this paper is to show that the answer is yes. Further, counterexamples of [Pycia and Troyan \(2023a\)](#) and [Basteck and Ehlers \(2024\)](#) show that our characterization is effectively tight: relaxing obvious strategy-proofness to strategy-proofness or relaxing symmetry to equal treatment of equals would break the characterization.

Our work is related to the literature that shows the equivalence between Random Priority and certain trading-style mechanisms based on Gale’s top trading cycles (TTC) algorithm. [Abdulkadiroğlu and Sönmez \(1998\)](#) and [Knuth \(1996\)](#) show that Random Priority is equivalent to the core from random endowments, which first randomly allocates objects to agents and then allows them to trade via TTC. [Pathak and Sethuraman \(2011\)](#) show further that Random Priority is also equivalent to Partitioned Random Priority and Partitioned Random Endowment. They also extend this analysis to school choice problems, in which schools can have multiple seats, and show that it is equivalent for a school district to use a common lottery to determine the priority ordering at all schools or multiple independent lotteries across schools.⁹ [Carroll \(2014\)](#) provides a more general equivalence of symmetrizations of [Pápai’s \(2000\)](#) hierarchical exchange mechanisms.¹⁰ While related, our results are distinct because we consider a different class of mechanisms, all obviously strategy-proof and efficient ones: not all efficient obviously strategy-proof mechanisms are trading mechanisms and trading mechanisms, while efficient, are in general not obviously strategy-proof.¹¹

Our analysis also contains methodological innovations that might be more generally useful. For instance, we show how to reduce the problem of characterizing symmetric mechanisms to the simpler problem of characterizing mechanisms that are obtained by symmetric randomizations over agents’ roles in a base mechanism, so called symmetrizations of the base mechanism. The remainder of our proof strategy builds at a high level on the bijective approach pioneered by [Abdulkadiroğlu and Sönmez \(1998\)](#) and [Knuth \(1996\)](#) and extended by [Carroll \(2014\)](#) to some mechanisms where a priority list for each object is given in terms of n abstract *roles*. The allocation is then determined by (i) randomly selecting a *role assignment function* that determines which agent is in which role and (ii) running the TTC algorithm under the selected role assignment. This can then be used to show there is a bijection between the role assignments and Random Priority orderings, which implies the mechanisms are equivalent. Our proof extends this role assignment approach to the class of obviously strategy-proof and Pareto efficient mechanisms, where the roles correspond to players in an extensive-form game. We use [Pycia and Troyan’s \(2023b\)](#) characterization of obviously strategy-proof mechanisms and results from [Bade and Gonczarowski \(2017\)](#) on Pareto efficient and obviously strategy-proof mechanisms to

starting with [Li \(2017\)](#)—also shows that correct play is indeed more prevalent in obvious strategy-proof mechanisms than merely strategy-proof ones.

⁹[Pycia \(2019\)](#) shows that symmetrizations of all [Abdulkadiroğlu and Sönmez \(2003\)](#) top trading cycle mechanisms are equivalent.

¹⁰[Bade \(2020\)](#) further extends it to [Pycia and Ünver \(2017\)](#)’s trading-cycle mechanisms.

¹¹[Li \(2017\)](#) shows that the classic TTC mechanism in which each agent begins by owning a unique object is not obviously strategy-proof. More general formulations of TTC where each object has a priority relation of the agents describing the “property rights” over the objects might be obviously strategy-proof in some cases. When each object has the exact same priority relation, TTC is trivially equivalent to RP (and thus is obviously strategy-proof), but more generally, [Troyan \(2019\)](#) and [Mandal and Roy \(2022\)](#) show that this cannot be extended very far: the class of priority structures for which TTC is obviously strategy-proof is restrictive, essentially requiring that at any point, “ownership” of all of the objects is divided among at most two agents.

obtain a representation of this class. Given the richness of the class of games we allow, our construction of the bijection from role assignments to orderings is more complex than the linear approach employed by the previous work. We first construct a (one-to-one) injection from role assignments to merely partial orderings, which we call “codings.” We show that for any coding, any way of breaking ties delivers a serial dictatorship ordering that results in the same outcome as the original mechanism. The bulk of the proof is then devoted to showing that there exists a way to break the ties and create complete orderings that preserves the one-to-one property, thereby constructing a bijection between role assignments and serial dictatorship orderings.

Following on our work, [Basteck \(2024\)](#) provided a second axiomatic characterization of Random Priority in finite markets relying on efficiency and symmetry, as well as a new axiom of probabilistic monotonicity that he introduced. Probabilistic monotonicity is an adapted version of Maskin monotonicity.

2. THE ALLOCATION PROBLEM

2.1. Environment

Let $\mathcal{N}$ be a set of **agents** and $\mathcal{X}$ a set of **objects**, where $|\mathcal{N}| = |\mathcal{X}|$ (in [Appendix D](#), we show that our main result continues to hold without this assumption). Each agent $i \in \mathcal{N}$ has a **strict preference relation**, $\succ_i$, over the set of objects $\mathcal{X}$, where for any $x, y \in \mathcal{X}$, $x \succ_i y$ denotes that object x is strictly preferred to object y . We also refer to $\succ_i$ as agent i 's **type**, and write $x \succeq_i y$ to denote that either $x \succ_i y$ or $x = y$. Let $\mathcal{P}$ denote the set of possible types, which consists of all possible strict rankings of the objects. We write $\succ_{\mathcal{N}} = (\succ_i)_{i \in \mathcal{N}}$ to denote a profile of types, one for each agent. A (deterministic) **allocation** $\mu : \mathcal{N} \rightarrow \mathcal{X}$ is a bijective function that assigns exactly one of the objects to each agent $i \in \mathcal{N}$. Let $\mathcal{M}$ be the set of deterministic allocations.

2.2. Extensive Game Forms

To determine the final allocation that will be implemented, the planner designs a **game form**, Γ . We consider imperfect-information, extensive game forms with perfect recall, which are defined in the standard way: There is a finite collection of partially ordered **histories**, $\mathcal{H}$. The notation $h' \subseteq h$ denotes that h' is a subhistory of $h \in \mathcal{H}$. We use $h_\emptyset$ to denote the initial history, and terminal histories are denoted with bars, $\bar{h}$. Each terminal history $\bar{h} \in \mathcal{H}$ is associated with some allocation in $\mathcal{M}$. At every non-terminal history $h \in \mathcal{H}$, one agent, denoted i_h , is called to play and chooses an **action** from a finite set $A(h)$. We allow for random moves by a non-strategic agent, Nature, who is not one of the agents in $\mathcal{N}$; at any history h at which Nature moves, it selects an action from $A(h)$ according to some predetermined probability distribution. We write $h' = (h, a)$ to denote the history that is reached by starting at h , and following the action $a \in A(h)$. To avoid trivialities, we assume that no agent moves twice in a row, and that $|A(h)| > 1$ for all non-terminal $h \in \mathcal{H}$. The set of histories at which an agent i (either in $\mathcal{N}$ or Nature) moves is denoted $\mathcal{H}_i = \{h \in \mathcal{H} : i_h = i\}$.

To capture imperfect information, $\mathcal{H}_i$ is partitioned into **information sets**, denoted $\mathcal{I}_i$. For any information set $I \in \mathcal{I}_i$ and $h, h' \in I$ and any subhistories $\hat{h} \subseteq h$ and $\hat{h}' \subseteq h'$ at which i moves, at least one of the following two symmetric conditions obtains: either (i) there is a history $\hat{h}^* \subseteq \hat{h}$ such that $\hat{h}^*$ and $\hat{h}'$ are in the same information set, $A(\hat{h}^*) = A(\hat{h}')$, and i chooses the same action at $\hat{h}^*$ and $\hat{h}'$, or (ii) there is a history $\hat{h}^* \subseteq \hat{h}'$ such

that $\hat{h}^*$ and $\hat{h}$ are in the same information set, $A(\hat{h}^*) = A(\hat{h})$, and i chooses the same action at $\hat{h}^*$ and $\hat{h}$. We denote by $I(h) \in \mathcal{I}_i$ the information set containing history h . Given two information sets I_1 and I_2 , if there exist $h_1 \in I_1$ and $h_2 \in I_2$ such that $h_1 \subseteq h_2$, then we write $I_1 \leq I_2$, and say that I_1 **precedes** I_2 , and that I_2 is a **continuation** of I_1 . With slight abuse of notation, we use $A(I)$ to denote the actions available at information set I . An object $x \in \mathcal{X}$ is **possible** for i at information set I if there is some $h \in I$ and some terminal history $\bar{h} \supseteq h$ such that at the allocation associated with $\bar{h}$, $\mu(i) = x$.

Given the agents' preferences over outcomes, the game form Γ determines an incomplete information extensive-form game, and, with slight abuse of terminology, in the sequel, we often refer to Γ simply as a “game” or “extensive-form game.”

2.3. Strategies, Mechanisms, and Equivalence

A **strategy** $S_i(>_i)$ for type $>_i$ of agent i specifies an action for each information set, $S_i(>_i)(I_i) \in A(I_i)$.¹² We use $S = ((S_i(>_i))_{>_i \in \mathcal{P}})_{i \in \mathcal{N}}$ to denote a profile of strategies. To avoid notational clutter, when the context is clear, we suppress the type-dependence of a strategy, and write $S_i(I_i)$ for the action chosen by agent i at I_i . A **mechanism** (Γ, S) is an extensive-form game Γ together with a profile of strategies, S . Given a profile of strategies and type profile $>_{\mathcal{N}}$, we say that an information set I is **on the path of play**, or **on path**, if there exists a strategy for Nature such that the path from the root of the game to the terminal history that is reached when agents follow $S(>_{\mathcal{N}})$ and Nature follows its strategy passes through I . For a type $>_j$ of agent j , we say that information set I is **on-path for** $>_i$ if there exists a type profile of the remaining agents $>_{\mathcal{N} \setminus \{j\}}$ and Nature such that I is on-path for $(>_i, >_{\mathcal{N} \setminus \{j\}})$.

Any mechanism induces a lottery over terminal histories, and thus, allocations. We say that two mechanisms (Γ, S) and (Γ', S') are **equivalent** if, for every profile of types $>_{\mathcal{N}}$, the distribution over allocations when agents follow S in Γ is the same as that when agents follow S' in Γ' . Every mechanism induces a mapping from type profiles to (random) allocations, which we call the **social choice rule** or the **direct mechanism**. If two mechanisms are equivalent, they implement the same social choice rule.

3. RANDOM PRIORITY AND ITS PROPERTIES

The Random Priority mechanism works as follows. Nature begins by first selecting an ordering of the agents uniformly at random from all possible agent orderings. Agents then move one at a time in this order, and each agent is given the opportunity to choose an object from a menu of all objects that are still available (i.e., that were not chosen by prior agents). At the end of the game, each agent is assigned to exactly one unique object, which determines the final allocation.

The Random Priority mechanism has desirable simplicity, efficiency, and fairness properties. Li (2017) shows that Random Priority is obviously strategy-proof.¹³ A strategy $S_i(>_i)$ is **obviously dominant** for player i (of type $>_i$) in game Γ if, for each on-path information set $I^* \in \mathcal{I}_i$, the worst possible outcome for i according to $>_i$ in the continuation game assuming i follows $S_i(I)$ at all $I \geq I^*$ is weakly preferred by i to the best possible

¹²We restrict attention to pure strategies. Allowing for mixed strategies would not change any of our results.

¹³Pycia and Troyan (2023b) show that it satisfies the even stronger simplicity standards of one-step simplicity (OSS) and strong obvious strategy-proofness (SOSP).

outcome for i in the continuation game if i plays some other action $a' \neq S_i(I^*)$.¹⁴ If there exists a profile of strategies S such that $S_i(>_i)$ is obviously dominant in Γ for all i and all $>_i$, then (Γ, S) is said to be **obviously strategy-proof (OSP)**. We then also say that the direct mechanism induced by (Γ, S) is obviously strategy-proof, thus recognizing that the existence of an obviously strategy-proof extensive-form allows designers to explain the direct mechanism in a simple way. Random Priority satisfies this criterion as, at an agent's turn, she is able to select from all remaining possible objects. Thus, the worst-case (and in fact, only) outcome from selecting her most preferred remaining object is getting this object, which is clearly at least as good as (and in fact strictly better than) selecting anything else.

Pareto efficiency and fairness of Random Priority have been recognized at least since [Abdulkadiroğlu and Sönmez \(1998\)](#). We say that a deterministic allocation is **Pareto efficient** if, given a type profile $>_{\mathcal{N}}$, no other allocation is weakly preferred by all agents, and strictly preferred by at least one; similarly, a mechanism (Γ, S) is *ex post* Pareto efficient (**Pareto efficient** for brevity) if it leads to a Pareto efficient allocation for all Nature's choices and agents' types; we then also say that the induced direct mechanism is Pareto efficient. Random Priority clearly satisfies this property: since each agent selects her most preferred remaining object at her turn, the only way to make an agent strictly better off is to give her an object that was taken by an earlier agent. But then this agent must be given an object taken by an even earlier agent, and so on. Eventually, one of these agents will be unable to be made better off, and so Random Priority is Pareto efficient.

We use a standard fairness criterion of symmetry: the mechanism treats agents equally in the sense that it would not change if any two agents i and j were to switch roles.¹⁵ More formally, a mechanism (Γ, S) is **symmetric** if, for any two agents $i, j \in \mathcal{N}$, the outcome distribution of the mechanism does not change when we transpose the types of agents i and j and at the same time transpose the objects the agents obtain. For instance, symmetry fails in a deterministic serial dictatorship in which i chooses first and j chooses second: if they have the same most preferred object $x \in \mathcal{X}$, then i obtains x in the original serial dictatorship; after transposing the types of i and j , i still obtains x , but after also transposing the received objects, i no longer obtains x , and so the mechanism is not symmetric. Random Priority, on the other hand, gives each ordering of the agents the same probability, and so in effect, the probability i obtains the preferred object is the same before and after the transposition.

4. THE MAIN RESULT

Random Priority succeeds on three important design dimensions: it is obviously strategy-proof, Pareto efficient, and symmetric. However, this is only a partial explanation of its success, as to now, it has remained unknown whether there exist other such mechanisms, and, if so, what explains the relative popularity of Random Priority over these alternatives. Our main result, [Theorem 1](#) provides an answer to this question: not only does Random Priority have good efficiency, fairness, and simplicity properties, its social choice function is the *only* direct mechanism that does so, thus explaining the widespread popularity of Random Priority in practice.

¹⁴[Li \(2017\)](#) presents the definition of obvious dominance in a slightly different way, using the notion of earliest points of departure. The two formulations are equivalent.

¹⁵In [Appendix A](#), we define the concept of roles, which make this informal definition formal and equivalent to the in-text definition. Because any permutation can be decomposed into a composition of transpositions, we can equivalently state the symmetry property as $\sigma^{-1} \circ (\Gamma, S) \circ \sigma = (\Gamma, S)$ for all permutations $\sigma : \mathcal{N} \rightarrow \mathcal{N}$.

THEOREM 1: *An obviously strategy-proof mechanism is symmetric and Pareto efficient if and only if it is equivalent to Random Priority.*

The proof of Theorem 1 can be found in the [Appendix](#) (with the proofs of some auxiliary lemmas relegated to the Supplemental Material, [Pycia and Troyan \(2026\)](#)). Here, we outline the main arguments.¹⁶ Given any arbitrary obviously strategy-proof mechanism, we transform it into an equivalent *millipede mechanism*. Millipede mechanisms are a class of mechanisms introduced by [Pycia and Troyan \(2023b\)](#).¹⁷ They are perfect information mechanisms in which Nature either moves once, at $h_\emptyset$, or Nature has no moves. Each time an agent $i \in \mathcal{N}$ is called to move, she is presented with some subset of objects she can take immediately, and leave the game; we refer to this as *clinging* an object, and the corresponding action as a *clinging action*.¹⁸ The agent may also be offered the opportunity to take a *passing action*. If an agent passes, she remains in the game, with the possibility of being offered additional objects for clinging in the future. A key feature of millipede mechanisms is that at each history, there can be at most one passing action. To satisfy obvious strategy-proofness, following a passing action, the mechanism must make certain guarantees to the agent in the continuation game that ensure she will never “regret” her decision to pass. [Pycia and Troyan \(2023b\)](#) show that millipede mechanisms are obviously strategy-proof and any obviously strategy-proof mechanism is equivalent to a millipede mechanism in which all agents follow greedy strategies, where a strategy is called *greedy* if, at any history h at which the agent moves, if the top object that is still possible for her at h is clinchable at h , the agent clinches the object; otherwise, the agent passes.¹⁹

The extensive-form game presented in Figure 1 (taken from [Pycia and Troyan \(2023b\)](#)) is an example of a millipede that allocates three objects x , y , and z to three agents, i_1 , i_2 , and i_3 . Agent i_1 moves first and can clinch either object x or y (and leave the game), or can pass (and remain in the game). If i_1 passes, agent i_2 can either clinch y (in which case the allocation is fully determined: i_1 receives z and i_3 receives x) or pass. Agent i_3 only

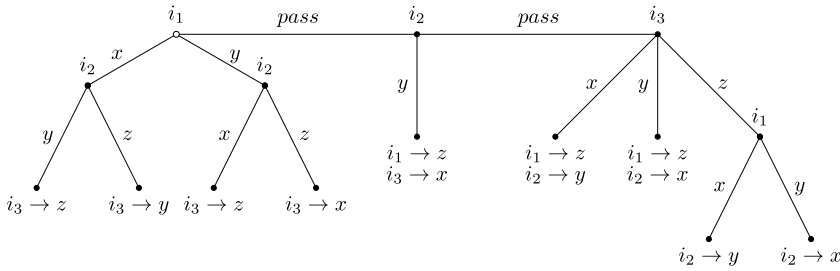


FIGURE 1.—An obviously strategy-proof and Pareto efficient game Γ with three agents and three objects.

¹⁶For $|\mathcal{N}| = 1$, Theorem 1 is immediate. For $|\mathcal{N}| = 2$, the equivalence is implied by Pareto efficiency when agents rank objects differently and it is implied by symmetry when they rank objects in the same way. [Bogomolnaia and Moulin \(2001\)](#) analyze the three-agent case, but their approach is different and, because of its reliance on ordinal efficiency, not applicable beyond three agents.

¹⁷[Pycia and Troyan \(2023b\)](#) define millipede mechanisms for a broad class of economic environments that includes the allocation problem we study. We adapt their definitions to our environment.

¹⁸At any history, there may be multiple clinching actions that clinch the same object for the agent who moves, but may affect the continuation game. An agent might also clinch an object—in the sense that irrespective of any further actions by any agent or nature the agent receives this object—at a history at which the agent does not move.

¹⁹We provide further details on millipede mechanisms in step 2 of the proof in [Appendix B](#).

moves following two passes, and at this point, i_3 can clinch any object. If i_3 clinches x or y , then the allocation is determined, and if agent i_3 clinches z , then i_1 can choose between x and y .

For an illustration of how the resulting millipede mechanism—the game and players’ greedy strategies—guarantee obvious strategy-proofness, consider agent i_1 . For all types of i_1 that rank x or y first, greedy strategies direct i_1 to clinch her top object at the first move, which is clearly obviously strategy-proof. Consider now any type of i_1 that ranks z first. At her first move, z is possible only following a pass, and therefore, any greedy strategy directs the agent to pass.²⁰ Notice that at all terminal histories following a pass, agent i_1 receives z (her top object), except if i_3 takes z at her move. In this latter case, following i_3 ’s clinching of z , the mechanism returns to agent i_1 , and offers her the opportunity to clinch either x or y . Thus, for i_1 , the worst-case outcome from passing is at least as good as anything she could have clinched at the initial move, and obvious strategy-proofness is satisfied.

The next step of the proof combines the structure of millipede mechanisms with properties of general Pareto efficient and obviously strategy-proof mechanisms identified by [Bade and Gonczarowski \(2017\)](#). Pareto efficiency means that every object must be possible for the first agent. If this agent clinches, then they are no longer active, and we enter a smaller subgame on the remaining agents and objects. When the first agent, say i , passes, the mechanism is able to infer that none of the objects offered for clinching is her top possible object, but otherwise, the possible rankings of the objects for the agent remain arbitrary. If two or more objects might be the agent’s favorite, then again by Pareto efficiency, all unclinched objects are possible for the next agent, say j (else, if we were to exclude an object x from j , it might be that j ranked x first while all other agents, including i , ranked it last, which would violate Pareto efficiency).

With two active agents for whom all objects are possible and whose top objects we do not know, all obvious dominance constraints can be simultaneously honored; with three or more active agents, it is generally no longer possible to do so. The exception to this is when we know which among possible objects is the most preferred for one of the currently active agents. The only way this may happen is if (cumulatively, at previous histories) the mechanism offered this agent for clinching everything possible for her except for this object; if the agent has passed on all objects possible for her except for one, then we can infer that the single object not yet offered is her favorite. Following [Bade and Gonczarowski \(2017\)](#), we refer to agents who have been offered every object possible for them except for one as *lurkers* and the unique object they have not been offered to clinch as a *lurked object*.²¹ When one of the currently active agents becomes a lurker, this allows the mechanism to introduce an additional active agent. Because for a lurked object we know that it is some currently active agent’s favorite, the mechanism can now exclude the newly active agent from ever receiving any lurked object without violating Pareto efficiency while at the same time respecting obvious dominance constraints for all of the active agents.

For example, consider again [Figure 1](#). At the first move, agent i_1 is offered x and y . By greedy strategies, if she passes, the mechanism is able to infer that i_1 ’s top choice is z . Upon passing, i_1 becomes a lurker for the object z , and the mechanism is able to exclude

²⁰ An object x is said to be *possible* for agent i at a history h if there is some terminal history $\bar{h} \succeq h$ at which i receives x .

²¹ Our formal definition of a lurker given in the [Appendix](#) is slightly more restrictive than theirs. See [Appendices B and C](#) for further details on lurkers and the role they play in obviously strategy-proof and Pareto efficient mechanisms.

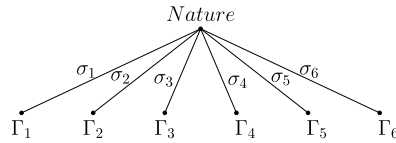


FIGURE 2.—The symmetrization of the mechanism from Figure 1. The mechanism begins with Nature drawing a role assignment function σ_k uniformly at random (there are $3! = 6$ possible permutations of the agents, and thus 6 role assignments). Then, the agents proceed to play the continuation game Γ_k , where Γ_k is the game from Figure 1 with the roles of the agents permuted according to σ_k . Each continuation game ($\Gamma_1, \dots, \Gamma_6$) is obviously strategy-proof and Pareto efficient. The grand mechanism (including the draw of the role assignment function) is obviously strategy-proof, Pareto efficient, and symmetric.

i_2 from receiving z , in effect making z possible for only i_1 and i_3 . Notice that this can be done while preserving Pareto efficiency: even though the mechanism has no information on i_2 or i_3 's preferences yet, if it turns out that i_3 's top choice is not z , the mechanism can safely give z to agent i_1 at that point without violating Pareto efficiency.

Pareto efficient millipede mechanisms need not be symmetric in general. Returning again to the mechanism whose game form is illustrated in Figure 1, consider the preference profile $\succ_N = (\succ_{i_1}, \succ_{i_2}, \succ_{i_3})$ defined by

$$\succ_{i_1} : x, y, z,$$

$$\succ_{i_2} : x, y, z,$$

$$\succ_{i_3} : z, y, x.$$

Under this profile, the outcome of the mechanism is $\{(i_1, x), (i_2, y), (i_3, z)\}$. If we transpose the preferences of i_1 and i_2 while at the same time transposing the objects agents i_1 and i_2 receive, the outcome is $\{(i_1, y), (i_2, x), (i_3, z)\}$, and thus symmetry fails. However, the mechanism can be symmetrized as follows. Let (Γ^*, S^*) be the mechanism shown in Figure 2. Game Γ^* begins with a move by Nature, drawing a permutation of players σ uniformly at random; we refer to such permutations σ as role assignments (we give a more formal definition of role assignments in Appendix A). The continuation game Γ_σ is isomorphic to Γ from Figure 1 with the agents permuted. For instance, if in the first step Nature draws the role assignment $\sigma_1(i_1) = i_1$, $\sigma_1(i_2) = i_2$, and $\sigma_1(i_3) = i_3$, then the agents continue by playing precisely the game in Figure 1; if instead Nature draws the role assignment $\sigma_2(i_1) = i_2$, $\sigma_2(i_2) = i_1$, and $\sigma_2(i_3) = i_3$, then the agents continue by playing the game Figure 1 except with the roles of i_1 and i_2 swapped. Similarly to how randomizing over deterministic serial dictatorships (which are not symmetric) produces the symmetric Random Priority mechanism, randomizing over role assignments in Γ produces the symmetric mechanism (Γ^*, S^*) .

In the proof, we show that it is sufficient to prove Theorem 1 for symmetrizations. We state this formally as the following:

PROPOSITION 1: *Suppose that, for every deterministic obviously strategy-proof and Pareto efficient perfect-information mechanism, its symmetrization is equivalent to Random Priority. Then, every symmetric, obviously strategy-proof and Pareto efficient mechanism is equivalent to Random Priority.*

Proposition 1 illustrates a general insight: because symmetrizations do not change symmetric mechanisms, to establish a property for all symmetric mechanisms from a class of

mechanisms, it is sufficient to establish it for symmetrizations of all mechanisms from this class.

By Proposition 1, it is sufficient to show that the symmetrization of every obviously strategy-proof and Pareto efficient mechanism is equivalent to Random Priority. In our example, we take the obviously strategy-proof and Pareto efficient mechanism (Γ, S) in Figure 1 and construct its symmetrization by having Nature randomly draw a role assignment and then the agents play the game Γ_k , which is equivalent to the game Γ from Figure 1 except the roles of the agents are permuted according to σ_k (see Figure 2).

Letting Σ be the set of all possible role assignment functions, we construct a mapping $f : \Sigma \rightarrow \text{Ord}$ between role assignment functions and serial dictatorship orderings such that (i) for each σ , the outcome of the continuation game $(\Gamma_\sigma, S_\sigma)$ is the same as a serial dictatorship in which agents choose in the order $f_\sigma(1), f_\sigma(2), \dots, f_\sigma(N)$ and (ii) f is a bijection.²² Since the symmetrized mechanism uniformly randomizes over role assignment functions, the probability of achieving any particular allocation μ is just the number of role assignment functions such that $(\Gamma_\sigma, S_\sigma)$ results in allocation μ , divided by $N!$. Similarly, because Random Priority uniformly randomizes over serial dictatorship orderings, the probability of achieving any particular allocation μ is just the number of serial dictatorship orderings that result in μ divided by $N!$. If there exists a bijection as just described, these two numbers will be equal for any μ , and hence, the distribution over allocations in the symmetrized mechanism is the same as in Random Priority, that is, the two mechanisms are equivalent.²³

At the core of the proof of Theorem 1 is our general construction of the bijection f and the proof that it is indeed a bijection. For sake of illustration, consider again the example with the preference profile $\succ_N$ given above.²⁴ Consider a role assignment function such that $\sigma(i_k) = i_k$ for $k = 1, 2, 3$. Under this role assignment, the game among the agents is that shown Figure 1, and the resulting play is as follows: agent i_1 moves first and clinches x ; agent i_2 moves second and clinches y ; agent i_3 receives z without being called to move. In this case, our bijection f maps σ to the following serial dictatorship ordering: $f_\sigma(1) = i_1$, $f_\sigma(2) = i_2$, $f_\sigma(3) = i_3$.

Both $(\Gamma_\sigma, S_\sigma)$ and a serial dictatorship with agent ordering f_σ result in the same outcome: $\{(i_1, x), (i_2, y), (i_3, z)\}$. If instead Nature draws the permutation $\sigma'(i_1) = i_2$, $\sigma'(i_2) = i_1$, $\sigma'(i_3) = i_3$, then the game path of $(\Gamma_{\sigma'}, S_{\sigma'})$ has agents i_2 , i_1 , and i_3 clinching x , y , and z (in this order). The associated serial dictatorship in this case is $f_{\sigma'}(1) = i_2$, $f_{\sigma'}(2) = i_1$, $f_{\sigma'}(3) = i_3$. Once again, it can be checked that both $(\Gamma_{\sigma'}, S_{\sigma'})$ and a serial dictatorship under agent ordering $f_{\sigma'}$ result in the same final allocation: $\{(i_1, y), (i_2, x), (i_3, z)\}$. Indeed, as we show below, any time the game Γ_σ starts with several agents choosing clinching moves, then we map it to a serial dictatorship that starts with the same agents moving in the same order, and it is easy to see that if all moves are clinching moves, $(\Gamma_\sigma, S_\sigma)$ always result in the same allocation as the corresponding serial dictatorship.

The mapping of game paths that involve passing is more subtle. In the present example, passing is on the game path if the role of i_1 is assigned to agent i_3 . There are two such

²² $f_\sigma \in \text{Ord}$ is an ordering of all of the agents in N such that $f_\sigma(j)$ is the j th agent in this ordering.

²³ The bijection idea was first employed by Abdulkadiroğlu and Sönmez (1998) and Knuth (1996), and has since been used by others (e.g., Pathak and Sethuraman (2011) and Carroll (2014)). Our construction is different from the bijections in the earlier literature, and it relies on the properties of Pareto efficient millipede games.

²⁴ The bijection is constructed for a fixed preference profile. Different preference profiles will lead to different bijections, but we still have the outcome distributions for the two mechanisms the same profile-by-profile, and thus the mechanisms are equivalent.

permutations: if $\sigma''(i_2) = i_2$, then the resulting outcome is $\{(i_1, x), (i_2, y), (i_3, z)\}$, and if $\sigma'''(i_2) = i_1$, then the resulting outcome is $\{(i_1, y), (i_2, x), (i_3, z)\}$.

To what serial dictatorships should we map these two permutations? In this simple example, it can be checked by hand that mapping σ'' into a serial dictatorship with agents ordered i_3, i_1, i_2 , and mapping σ''' to a dictatorship with agents ordered i_3, i_2, i_1 achieves the desired bijection. However, in general, there is no simple rule of thumb in mapping role assignment functions that entail passing on the path of play: notice that in the present example, the resulting serial dictatorships do not order agents in the order in which they move in Γ_σ , nor do they order agents in the order in which they take their objects.

We proceed by dividing any run of the Pareto efficient millipede mechanism (under a specific role assignment and preference profile) into “rounds”; a round consists of a sequence of consecutive moves that determine the assignment of some (endogenously determined) number of lurked objects and one or two unlurked objects. The agents assigned these objects never move after the end of the round, and the continuation mechanism (the continuation game and strategies) after the round ends is again a Pareto efficient millipede for fewer agents and objects. The division into rounds hence has a recursive structure. We order the agents assigned in earlier rounds above those assigned in later rounds, and within a round, agents who receive lurked objects are ordered according to the order in which the object they receive became lurked. This ordering is not necessarily consistent with the order in which agents move or become lurkers, as a lurker might end up with an object different than the one she lurks. A key obstacle in the proof is that the information from the round is sometimes not sufficient to order agents who receive unlurked objects. We deal with this issue by initially having such agents “tie”. We then show that the mapping from role assignments to such partial orderings (with ties allowed) is injective, and that no matter how the ties are broken, the resulting serial dictatorship ends up with all agents receiving the same allocation as in the arbitrary Pareto efficient and obviously strategy-proof mechanism. The bulk of the proof is devoted to showing that it is possible to break these ties—and thus, create a mapping to complete serial dictatorship orderings—in a way that preserves the injectivity property, which establishes that the final mapping from role assignments to orderings of agents is a bijection. The full details of this construction can be found in the [Appendix](#).

5. CONCLUSION

We have resolved in the positive the long-standing conjecture about Random Priority: it is the unique mechanism satisfying desirable incentive, efficiency, and fairness properties. This characterization provides an explanation for the popularity of Random Priority. The characterization also implies that imposing more restrictive simplicity standards than obvious strategy-proofness—e.g., one-step simplicity or strong obvious strategy-proofness of [Pycia and Troyan \(2023b\)](#)—imposes no further limitations on Pareto efficient and symmetric mechanisms in the house allocation environment.

APPENDIX A: ROLES AND ROLE ASSIGNMENT FUNCTIONS

Our terminology of roles and role assignment generalizes [Carroll’s \(2014\)](#) terminology of priority roles in [Pápai \(2000\)](#)’s hierarchical exchanges to general extensive-form games. In the definition of our fairness axiom and the proof of the main theorem below, we make use of the concepts of roles and role assignment functions, which we introduce here. Let $\mathcal{R}$ be a set of players such that $|\mathcal{R}| = |\mathcal{N}|$; we call each $r \in \mathcal{R}$ a **role**. Given any game

Γ , we define a corresponding **proto-mechanism**, $(\tilde{\Gamma}, \tilde{S})$, which consists of a **proto-game**, $\tilde{\Gamma}$, and a profile of **proto-strategies**, $\tilde{S}$. The proto-game $\tilde{\Gamma}$ is equivalent to Γ , except that each history h is assigned to a particular role $r \in \mathcal{R}$ (rather than an agent in $\mathcal{N}$), with the restriction that if two histories are controlled by the same agent in Γ , then they are controlled by the same role in $\tilde{\Gamma}$. Formally, letting $\rho : \mathcal{H} \rightarrow \mathcal{R}$ be the function that maps each history h to the role that moves at h in $\tilde{\Gamma}$, we require that $\rho(h) = \rho(h')$ if and only if $i_h = i_{h'}$ in Γ . The proto-strategy profile $\tilde{S} = (\tilde{S}_r)_{r \in \mathcal{R}}$ is defined such that $\tilde{S}_r = S_i$, where r is the role that controls the same histories in $\tilde{\Gamma}$ that are controlled by agent i in Γ .

Let Σ be the set of bijections $\sigma : \mathcal{R} \rightarrow \mathcal{N}$ between the set of roles and the set of agents $\mathcal{N}$; we call these bijections **role assignment functions**. Given a proto-mechanism $(\tilde{\Gamma}, \tilde{S})$, each role assignment function $\sigma \in \Sigma$ determines a mechanism for the agents in $\mathcal{N}$, denoted $(\Gamma_\sigma, S_\sigma)$, as follows: Γ_σ is the extensive-form game with the same game tree as the proto-game $\tilde{\Gamma}$, and such that at each non-terminal history h , the agent called to move is $\sigma(\rho(h))$; at each terminal history in Γ_σ , the object assigned to agent i is the same as the object assigned to role $\sigma^{-1}(i)$ in $\tilde{\Gamma}$; the strategy S_i of agent i in Γ_σ is the same as the strategy of role $\sigma^{-1}(i)$ in $(\tilde{\Gamma}, \tilde{S})$. There are $|\Sigma| = N!$ possible mechanisms $(\Gamma_\sigma, S_\sigma)$; we call them the permuted mechanisms. (See Section 4 for an example of how role assignments work.)

Given a mechanism (Γ, S) , we further define the **symmetrization** of (Γ, S) , denoted (Γ^*, S^*) , to be the following random mechanism: first, Nature chooses a role assignment function σ uniformly at random from the set of all possible role assignment functions, and then, the agents play Γ_σ with strategies S_σ .²⁵

APPENDIX B: PROOF OF THEOREM 1

We break the proof down into six steps. Step 1 shows that it is sufficient to consider symmetrized mechanisms. Steps 2 and 3 show that we can further restrict attention to a subset of the class of millipede mechanisms of [Pycia and Troyan \(2023b\)](#). Step 4 constructs a coding algorithm that maps each of the permuted mechanisms $(\Gamma_\sigma, S_\sigma)$ that make up the symmetrization into a partial ordering of the agents that might include ties. Step 5 shows that for any tie-breaking of the partial ordering produced by the coding algorithm, a serial dictatorship run under this agent ordering produces the same allocation as the $(\Gamma_\sigma, S_\sigma)$. Step 6 then shows that it is possible to break ties in such a way that the final mapping is in fact a bijection between permuted mechanisms and serial dictatorship orderings.

In the proof, we use the concept of roles and role assignments introduced in Appendix A. Proofs of some intermediate results not given here are found in Appendix C.

Step 1: Symmetrization Reduction

The first step in proving Theorem 1 is to recognize that it is sufficient to prove the theorem for any symmetrization over Pareto efficient deterministic mechanisms. This is so

²⁵While this construction implies that different agents play the same strategies in the same role, our arguments only rely on the weaker assumption that an agent's strategy $S_{\sigma,i}(>_i)$ depends only on her own preferences and her role assignment, and not on the roles assigned to other agents. In other words, in any two subgames Γ_A and Γ_B following Nature's selection of role assignments σ_A and σ_B , if $\sigma_A^{-1}(i) = \sigma_B^{-1}(i) = r_n$, then $S_{A,i}(>_i)(h_A) = S_{B,i}(>_i)(h_B)$ for any equivalent histories h_A and h_B in these two games. Note that this element of the construction relies on the fact that the strategies are dominant; that is, they remain optimal regardless of strategies played by other agents.

because every symmetric mechanism can be expressed as a lottery over symmetrizations. If each of these symmetrizations is equivalent to Random Priority, then so is the lottery over them. We stated this insight as Proposition 1 above and we prove it now.

PROOF OF PROPOSITION 1: Take a symmetric, obviously strategy-proof, and Pareto efficient mechanism (Γ, S) . Lemma A.4 of [Pycia and Troyan \(2023b\)](#) shows that for every obviously strategy-proof mechanism, there is an equivalent obviously strategy-proof mechanism with perfect information in which Nature moves at most once, as the first mover.²⁶ Thus, it is without loss of generality to assume that (Γ, S) has perfect information and that Nature moves only at the beginning of the game. Because (Γ, S) is symmetric, its symmetrization (Γ^*, S^*) is equivalent to (Γ, S) . Furthermore, (Γ^*, S^*) is a lottery over symmetrizations of each deterministic perfect-information continuation game Γ' after Nature's move in (Γ, S) . The mechanism given by game Γ' , together with the strategy profile induced from Γ , is obviously strategy-proof and Pareto efficient, and hence by the assumption of the proposition it is equivalent to Random Priority. Because every lottery over Random Priority lotteries is still equivalent to Random Priority, the proposition obtains. *Q.E.D.*

In light of the above proposition, it is sufficient to prove Theorem 1 for symmetrizations, that is, it is sufficient to prove the following.

PROPOSITION 2: *Let (Γ, S) be an obviously strategy-proof and Pareto efficient deterministic perfect-information mechanism. Then, the symmetrization of (Γ, S) is equivalent to Random Priority.*

Steps 2–6 are devoted to showing Proposition 2, which, combined with Proposition 1, proves Theorem 1.

Step 2: Millipede Reduction

Let (Γ, S) be an obviously strategy-proof and Pareto efficient deterministic perfect-information mechanism. The first step in the proof of Proposition 2 shows that it is without loss of generality to assume that (Γ, S) is a millipede mechanism. Millipede mechanisms are a class of mechanisms introduced in [Pycia and Troyan \(2023b\)](#), who show that, in a broad class of preference environments that include our setting, any obviously strategy-proof mechanism is equivalent to a millipede mechanism. Broadly speaking, in our environment, a millipede mechanism is a perfect-information, extensive-form game such that at each history, one agent is called to move and is presented with a menu of objects. If the agent i who moves at some h chooses to take object x from the menu, then they receive x , and leave the game (never move again after h); in this case, we say that i has *clinched* x , and call the action taken by i at h a *clinching action*. At h , the agent may also be offered an opportunity to pass. If the agent passes, the object they receive is not yet determined and they might move again, waiting for better clinching options in the future.

To formally define a millipede game, we need the following definitions, which are adapted from [Pycia and Troyan \(2023b\)](#):

²⁶ [Ashlagi and Gonczarowski \(2018\)](#) briefly mention this result in a footnote while [Mackenzie \(2020\)](#) extends Lemma A.4 to extensive-form games without perfect recall.

- Possible objects: Recall that object x is **possible** for agent i at history h if there is a terminal history $\bar{h} \supseteq h$ at which i receives x . We let $P_i(h)$ denote the set of objects that are possible for i at h . Object x **becomes impossible** for i at h if $i_h = i$ or h is terminal, $i_{h'} = i$ for some $h' \subsetneq h$, and $x \in P_i(h')$ for all $h' \subsetneq h$ such that $i_{h'} = i$, but $x \notin P_i(h)$.
- Clinchable objects: Object x has been **clinched** by agent i at history h if i receives x at all $\bar{h} \supseteq h$. Object x is **clinchable** for agent i at history h if i moves at h and there is some action $a \in A(h)$ such that i has clinched x at $h' = (h, a)$. We let $C_i(h)$ denote the set of objects that are clinchable for agent i at h .
- Clinching actions: An action $a \in A(h)$ is called a **clinching action** if agent i_h (the agent who moves at h) has clinched x at history $h' = (h, a)$.
- Passing actions: Any action $a \in A(h)$ that is not a clinching action is a **passing action**.

REMARK 1: In the above definitions, note the distinction between a “clinching action” and an object being “clinched.” In particular, it is possible for an agent i to have clinched an object without being called to actively take a clinching action.²⁷ Note also that at any history h at which i moves, for any object $x \in C_i(h)$, there might be multiple actions $a \in A(h)$ that clinch x .

At a terminal history $\bar{h}$, no agent is called to move and there are no actions. However, at terminal histories $\bar{h}$, it is notationally convenient to use the notation $i_{\bar{h}}$ to refer to any agent $i \in \mathcal{N}$ and to define $C_i(\bar{h}) = \{x\}$, where x is the object that i receives at $\bar{h}$. We further define the following pieces of notation:

- $C_i^{\subseteq}(h)$ is the set of objects that have been previously clinchable for i at some subhistory of h ; formally, $C_i^{\subseteq}(h) = \{x : x \in C_i(h') \text{ for some } h' \subseteq h \text{ s.t. } i_{h'} = i\}$.
- $C_i^{\subsetneq}(h)$ is the set of objects that have been previously clinchable for i at some *strict* subhistory of h ; formally, $C_i^{\subsetneq}(h) = \{x : x \in C_i(h') \text{ for some } h' \subsetneq h \text{ s.t. } i_{h'} = i\}$. If $x \notin C_i^{\subsetneq}(h)$, then we say x is **previously unclinched** at h .

With these definitions, a **millipede game** is a finite extensive-form game of perfect information that satisfies the following properties:

- (M1) Nature either moves once, at the empty history $h_{\emptyset}$, or Nature has no moves.
- (M2) At any history at which an agent moves, all but at most one action are clinching actions, and following any clinching action, the agent does not move again.
- (M3) At all h , terminal and non-terminal, if there exists a previously unclinched object x that becomes impossible for agent i_h at h , then $C_{i_h}^{\subseteq}(h) \subseteq C_{i_h}(h)$.

Given a mechanism (Γ, S) and a type $\succ_i$, a strategy $S_i(\succ_i)$ is a **greedy strategy** if, at any history $h \in \mathcal{H}_i$, it satisfies the following: if the $\succ_i$ -best still-possible object in $P_i(h)$ is clinchable at h , then $S_i(\succ_i)(h)$ clinches this object; otherwise, $S_i(\succ_i)(h)$ is the passing action. A **millipede mechanism** is a millipede game with a profile of greedy strategies. In a millipede mechanism, it is obviously dominant for an agent to clinch the best possible object at h whenever it is clinchable. Condition (M3) of the millipede definition says that when some previously unclinched object becomes impossible for an agent, the next time she moves,

²⁷This distinction matters in our analysis of efficient and obviously strategy-proof games (efficient millipedes). In these games, clinching without being called to move can happen when an agent i has been offered to clinch and has passed on all possible objects for them except for one, x . This implies that x is i 's favorite object that was ever possible and might allow the mechanism to later assign x to i without calling i to move (depending on the actions of other agents). A specific example of this arises at histories satisfying property 5 of efficient millipedes that we describe in the sequel.

she is offered at least a clinching action for each object that was previously clinchable. This ensures that an agent never “regrets” her decision to pass on a previously offered object, and is formally what is needed to guarantee passing at h is obviously dominant when an agent’s best possible object at h is not clinchable. An example of a millipede mechanism in a three-agent, three-object setting is given in Figure 1.

LEMMA 1—Pycia and Troyan (2023b): *Every obviously strategy-proof mechanism is equivalent to a millipede mechanism.*

Using the above result from Pycia and Troyan (2023b), it is without loss of generality to assume in Proposition 2 that (Γ, S) is a millipede mechanism. Further, a mechanism (Γ, S) is said to be **pruned** if every history is on the path of play for some type profile (when information set I is on path for type profile $\succ_N$ or type $\succ_i$, we also say that $\succ_N$ or $\succ_i$ **reaches** I). By Li (2017)’s Pruning Principle, it is without loss of generality to assume that all millipedes (and obviously strategy-proof mechanisms in general) discussed below are pruned. Thus, to prove Proposition 2, we must show that the symmetrization of any Pareto efficient and pruned millipede mechanism is equivalent to Random Priority.

Step 3: Pareto Efficient Millipedes

Obvious strategy-proofness allows us to assume that (Γ, S) is a pruned millipede mechanism, by Lemma 1. Adding Pareto efficiency allows us to further restrict attention to a subclass of millipede mechanisms that we describe in this step. To describe this class, we must first introduce the concept of a lurker, which is a modification of a similar concept in Bade and Gonczarowski’s (2017, hereafter BG) analysis of Pareto efficient obviously strategy-proof mechanisms. Informally, a lurker is an agent who has previously been offered a clinching action for each object that is possible for her except for one, which she is said to “lurk.”

Consider a Pareto efficient millipede mechanism (Γ, S) . Call an agent i **active** at h if she has been previously called to play at some $h' \subseteq h$, and has not yet clinched an object at h .²⁸ Let $A(h)$ denote the set of active agents at h . Recall that $C_i^{\subseteq}(h)$ is the set of objects agent i has been offered to clinch at some subhistory of h and $C_i^{\subset}(h)$ is the set of objects agent i has been offered to clinch at some strict subhistory of h . Further, define $G_i(h)$ as the set of objects that are **guaranteeable** for i at h ; formally, $x \in G_i(h)$ if and only if there exists a continuation strategy S_i such that i receives object x at all terminal histories $\bar{h} \supseteq h$ that are consistent with i following strategy S_i starting from h .²⁹

We define lurkers recursively as follows. At the empty history, no agent is a lurker. Consider a history h and an agent i . If i has moved at a strict subhistory of h , then let $h' \subsetneq h$ be the maximal strict subhistory such that $i_{h'} = i$. Agent i is said to be a **lurker**

²⁸An agent might still be called to play at a history h at which she has clinched an object. At this point, the agent may still influence the continuation game, but can no longer actively influence the object they themselves receive, and so formally, we do not classify them as active. See footnote 37 for the context in which this may occur.

²⁹Note the distinction between guaranteeable objects, $G_i(h)$, and clinchable objects, $C_i(h)$: informally, an object x is clinchable at h if there is action $a \in A(h)$ such that i receives x “immediately” (and so no other objects are possible for i following action a), whereas if x is guaranteeable at h , there may be other objects that are possible, but there is some continuation strategy such that if i sticks to this strategy in the continuation game, she can guarantee she will receive x , no matter what the other agents do. The concepts of active agents and guaranteeable objects were introduced in Pycia and Troyan (2023b).

for object x at h if (i) $P_i(h) \neq G_i(h)$, (ii) $x \in P_i(h')$, (iii) $C_i^{\subseteq}(h') = P_i(h') \setminus \{x\}$, and (iv) $x \notin C_j^{\subseteq}(h')$ for any other $j \neq i$ that is active at h' and is not a lurker at h' . Otherwise, i is not a lurker at h . Thus, i is not a lurker for any object x at h if i has not moved at any strict subhistory of h or if i has moved at some strict subhistory but at least one of the conditions (i)–(iv) fails.³⁰ If some agent i is a lurker for an object x at a history h , then we say x is a **lurked object** at h . We use the term **BG lurker** to refer to any agent that satisfies (i), (ii), and (iii).³¹ By conditions (ii) and (iii), each lurker or BG lurker lurks only one object, and [Bade and Gonczarowski \(2017\)](#) show that each BG-lurked object has only one BG lurker. The analogous claim holds for lurked objects; see Lemma 11. BG also show that, at any history, at most two active agents are not BG lurkers. Our Lemma 12 shows that this also continues to hold for our definition of lurkers.³²

At any h , we partition the set of active agents as $\mathcal{A}(h) = \mathcal{L}(h) \cup \tilde{\mathcal{L}}(h)$. The set $\mathcal{L}(h) = \{\ell_1^h, \dots, \ell_{\lambda(h)}^h\}$ is the set of lurkers and $\tilde{\mathcal{L}}(h)$ is the set of active non-lurkers, where $\lambda(h) = |\mathcal{L}(h)|$ denotes the number of lurkers at h . Let $\mathcal{X}(h)$ denote the set of still-available (unclined) objects at h , and partition this set as $\mathcal{X}(h) = \mathcal{X}^{\mathcal{L}}(h) \cup \tilde{\mathcal{X}}^{\mathcal{L}}(h)$, where $\mathcal{X}^{\mathcal{L}}(h) = \{x_1^h, \dots, x_{\lambda(h)}^h\}$ is the set of lurked objects and $\tilde{\mathcal{X}}^{\mathcal{L}}(h) = \mathcal{X}(h) \setminus \mathcal{X}^{\mathcal{L}}(h)$ is the set of unlurked objects at h . We order the sets so that agent ℓ_m^h lurks objects x_m^h , and if $m' < m$, then lurker $\ell_{m'}^h$ is **older** than lurker ℓ_m^h , in the sense that $\ell_{m'}^h$ first became a lurker for $x_{m'}^h$ at a strict subhistory of the history at which ℓ_m^h became a lurker for x_m^h ; we also say that lurker ℓ_m^h is **younger** than lurker $\ell_{m'}^h$. For BG lurkers, we use the same older and

³⁰An agent might be a lurker at a history h but no longer be a lurker at a continuation history $h'' \supsetneq h$ at which condition (i) no longer holds because the agent has clinched an object by h'' . When the context is clear, for simplicity we might refer to such agents as lurkers, with the understanding that we are referring to the agents who were lurkers at h . See footnote 37 for the context in which we do so.

³¹BG lurkers were studied in [Bade and Gonczarowski \(2017\)](#), and we keep the term lurker for the redefined concept as an acknowledgment of their work. Because we impose condition (iv), our definition of a lurker is more restrictive than their Definition E.9: all lurkers in our sense are BG lurkers, but the converse need not hold. In consequence, our definition of a non-lurker is more permissive: a non-lurker in our usage may fail to be a BG non-lurker. We include (iv) in the definition of a lurker to facilitate the construction of our coding algorithm in step 4 that maps role assignment functions to agent orderings; our coding algorithm treats BG lurkers who do not satisfy (iv) the same as other non-lurkers and differently from how it treats lurkers.

³²While the formal proofs (provided in Appendix C) require more care, here is a rough intuition why, at any history, there might be at most two active non-lurkers and why no two lurkers may lurk the same object. Pareto efficiency requires that everything is possible for the first agent to move. If this first mover clinches an object, they stop being active. The structure of the millipede games implies that a mover continues to be active only when they pass, and a greedy strategy means that we can only establish that one of the objects not yet offered for clinching is the top possible object; the ranking of other objects can then remain arbitrary. If on path following the pass, two or more objects might be the top choice for the passing agent, say i , then Pareto efficiency implies that all not yet clinched objects are possible for the next agent, say j , that moves for the first time (else, if we exclude object a from j , we have no guarantee that j wanted a as their top choice while all other agents, including i , rank a as their worst object, in which case the exclusion would violate Pareto efficiency). With two active agents for whom all objects are possible and whose top objects we do not know, we can simultaneously honor all of their obvious dominance constraints. However, with three or more such active agents, it is no longer possible to simultaneously honor all of these constraints. Becoming a lurker roughly means that the mechanism has “learned” that i ’s favorite object that was ever possible for her is the one that she lurks, x . This allows the mechanism to exclude other agents from ever receiving x without violating Pareto efficiency, and is what allows the introduction of a third active agent while still being able to satisfy obvious dominance. If one of these agents becomes a lurker, then a fourth agent can become active, etc. In effect, we might have an arbitrary number of lurkers, but at most two active agents who are not lurkers.

younger comparisons and analogous notation ($\mathcal{L}^{\text{BG}}(h)$, $\tilde{\mathcal{L}}^{\text{BG}}(h)$, $\mathcal{X}^{\mathcal{L}, \text{BG}}(h)$, $\tilde{\mathcal{X}}^{\mathcal{L}, \text{BG}}(h)$, and $\lambda^{\text{BG}}(h)$).³³

As agents continue to take successive passing actions, the set of lurkers and the set of lurked objects continue to grow, until eventually, we reach a history h where some agent i clinches some object x .³⁴ Further, by Lemma 6 in Appendix C, any agent i who moves at a history h immediately following a passing action (that is, $h = (h', a^*)$, where h' is the immediate predecessor of h) is not a lurker at h . When an agent i clinches an object x at some such h , then we can then determine the assignments of all lurkers at h (if there are any) as follows:

- If $x \in \tilde{\mathcal{X}}^{\mathcal{L}}(h)$, each lurker $\ell_m^h \in \mathcal{L}(h)$ receives her lurked object, x_m^h .
- If $x = x_{m_1}^h$ for some lurked $x_{m_1}^h \in \mathcal{X}^{\mathcal{L}}(h)$, then all older lurkers $\ell_{m'}^h$ for $m' < m_1$ receive their lurked objects $x_{m'}^h$; lurker $\ell_{m_1}^h$, whose lurked object is assigned to i , receives her favorite object from the set $\mathcal{X}(h) \setminus \{x_1^h, \dots, x_{m_1}^h\}$.
 - If $\ell_{m_1}^h$ is assigned an unlurked object, then all remaining lurkers get their lurked objects; if $\ell_{m_1}^h$ is assigned a lurked object $x_{m_2}^h$ for some $m_2 > m_1$, then all older unmatched lurkers ($\ell_{m'}^h$ for $m_1 < m' < m_2$) receive their lurked objects. Lurker $\ell_{m_2}^h$ gets his favorite object from $\mathcal{X}(h) \setminus \{x_1^h, \dots, x_{m_2}^h\}$.
 - This process is repeated until some lurker ℓ_m^h receives an unlurked object, at which point all remaining unassigned lurkers are assigned their lurked objects.

These assignments follow from Lemma 16 in Appendix C (which in turn is derived from Lemma E.17 in BG, which shows these assignments hold for BG lurkers). Notice that in the above assignments, there is a unique active agent j who is assigned an unlurked object y ; this agent might be the agent i who started the chain of assignments by clinching at h , or one of the lurkers later in the chain. Because there can be at most two non-lurkers at any h , one of whom was the agent i who moves at h , there might be at most one additional active agent, j' , who is not part of the above assignments. If such a j' exists and $y \in C_j^{\subseteq}(h)$, then j' receives her favorite object among objects that are possible for her after the above assignments are carried out.³⁵ (If $y \notin C_{j'}^{\subseteq}(h)$, then the assignment of j' is not yet settled, and will be determined later in the mechanism.)

As we formalize in Lemma 2 below, step 2 and the above structure of assignments allow us to assume that our base game Γ is a millipede game that has the following properties:³⁶

1. At each history h , there is at most one passing action in $A(h)$; this action, if it exists, is denoted $a^* \in A(h)$. With slight abuse of notation, when the context is clear, we

³³The ordering of the lurkers by age is strict because an agent becomes a lurker immediately after taking a passing action at a history by which she has (cumulatively) been offered a clinching action for each object possible for her except for one.

³⁴Lemma 7 in Appendix C shows that once an agent becomes a lurker, she remains a lurker at any continuation history following a sequence of passing actions: that is, if $h' = (h, a^*, \dots, a^*)$, then $\mathcal{L}(h) \subseteq \mathcal{L}(h')$.

³⁵This follows by property (M3) of millipede games. Intuitively, $C_j^{\subseteq}(h) = C_{j'}^{\subseteq}(h)$ because j' (being different from i) does not move at h . Hence, $y \in C_j^{\subseteq}(h)$ means that j' passed on y at some $h' \subsetneq h$. By greedy strategies, passing implies that y is not the agent's top possible choice, but y might still be the second-best object for j' . Once y is taken by agent j , the mechanism must guarantee j' gets her top choice, else it would not be obviously dominant for her to pass on y at h' . For a formal proof, see Lemma 17 in Appendix C.

³⁶Property 1 is the basic structure of millipedes presented in step 2; we restate it in order to introduce the a^* notation. That, without loss of generality, we can assume properties 2 and 3 is established in the proof of the millipede theorem of Pycia and Troyan (2023b). We can assume property 4 because by property 2 and greedy strategies, any passing move at h can be pruned in the manner of Li (2017)'s Pruning Principle for obviously strategy-proof games.

use the symbol a^* to represent the unique passing action at any history h (if such an

action exists), and write $h' = (h, \overbrace{a^*, \dots, a^*}^{n \text{ times}})$ to denote that history h' is the superhistory of h that is reached by starting at h and following n passing actions in a row; since there is at most one passing action at any given history, h' is uniquely defined.

2. If i moves at h and $x \in G_i(h)$, then there exists a clinching action $a_x \in A(h)$ that clinches x for i .
3. At any non-terminal history h , if there exist one or more agents such that $P_i(h) = G_i(h)$, then one of these agents moves at h .
4. If i moves at h and $P_i(h) = G_i(h)$, then $C_i(h) = P_i(h)$, there is no passing action at h , and i is not called to move at any $h' \supsetneq h$.
5. At any history (h, a^*) that ends with a passing action, the agent who moves is not a lurker. At any history (h, a_x) that ends with a clinching action (where x is the object clinched by action a_x):
 - (a) If x is a lurked object at h , all lurkers whose lurked object is strictly older than x have clinched their lurked object at (h, a_x) . Such older lurkers may or may not be called to move following (h, a_x) .³⁷ If any such lurkers are called to move, they do so in order from oldest to youngest lurker, and we let h' denote the history that ends with the clinching action of the youngest of these lurkers; otherwise, we let $h' = (h, a_x)$. None of the lurkers who have clinched their object at h' is called to move at or following h' . At h' , the agent who lurked x at h is called to move. This agent is offered for clinching each object that has not been clinched by h' and there is no passing action.
 - (b) If x is not a lurked object at h , then all lurkers at h have clinched their lurked object at (h, a_x) . The lurkers at h may or may not be called to move following (h, a_x) .³⁸ If any such lurkers are called to move, they do so in order from oldest to youngest lurker, and we let h' denote the history that ends with the clinching action of the youngest of these lurkers; otherwise, we let $h' = (h, a_x)$. None of the lurkers at h is called to move at or following h' .
 If there exists an agent j' who was active at h and has not yet clinched an object at h' , then j' moves at h' and:³⁹
 - (i) If $x \in C_{j'}^{\subseteq}(h)$, then j' is offered a clinching action for each object that remains possible for her.⁴⁰ There is no passing action.
 - (ii) If $x \notin C_{j'}^{\subseteq}(h)$, then j' is offered at least a clinching action for each object in $C_{j'}^{\subseteq}(h)$; she may also have other clinching actions and/or a passing action.⁴¹

³⁷The moment x is clinched at (h, a_x) , all of these older lurkers have clinched their lurked objects without being called to move (see Remark 1 on the distinction between clinching an object and a clinching action). These agents are thus no longer “active” (in the sense that they can no longer actively influence their own object; cf. footnote 28), but might still be called to move once more and can influence the continuation game. These agents are also no longer lurkers, but with some abuse of terminology, we continue to refer to them as lurkers when the context is clear that we are referring to agents who were lurkers at h (cf. footnote 30).

³⁸See footnote 37.

³⁹This agent j' is not a lurker because of the first sentence of 5(b). Furthermore, there can be at most one such agent j' . This follows as Lemma 12 tells us that there are at most two active non-lurkers at any history following a sequence of passes, and the above structure implies that once a clinching action is taken by some non-lurker $i \neq j'$ at such a history, only lurkers are called to move (and thus no new agents become active) until the point at which all active lurkers have clinched an object.

⁴⁰See footnote 35.

⁴¹By definition, j' is not a lurker, and as we show in Appendix C, her set of previously clinchable objects $C_{j'}^{\subseteq}(h)$ cannot contain any lurked objects (note that $C_{j'}^{\subseteq}(h) = C_{j'}^{\subseteq}(h)$ as j' does not move at h). Since in this

If no such j' exists, then all agents who were active at h have clinched an object by h' . If there remain agents who have not yet clinched an object, then one of these agents moves at h' , and a continuation game begins among the remaining unmatched agents and objects. Otherwise, the game ends.

We summarize the above discussion in the following lemma, the proof of which can be found in Appendix C.1.

LEMMA 2: *Every obviously strategy-proof and Pareto efficient mechanism (Γ, S) is equivalent to a pruned millipede mechanism satisfying properties 1–5.*

REMARK 2—Recursive Structure: Lemma 2 implies that any Pareto efficient millipede mechanism is equivalent to one with a recursive structure that proceeds in $k = 1, \dots, K$ rounds, with each round divided into two phases. We call these phases a *passing phase* and a *clinging phase*. Let $h_\emptyset^k$ denote the **initiating history** of round k . For $k = 1$, $h_\emptyset^k = h_\emptyset$, the root of the game; for $k > 1$, $h_\emptyset^k$ will be defined recursively, as the conclusion of the previous round.

Passing phase:

- Starting from $h_\emptyset^k$, agents take some number (possibly zero) of passing actions. At any $h' = (h_\emptyset^k, a^*, \dots, a^*)$ in the passing phase:
 - The agent called to move is not a lurker at h' . (Lemma 6)
 - There is at most one BG lurker who is not a lurker at h' , and if such an agent exists, there is no passing action at h' .⁴² (Lemma 10)
 - There are at most two active non-lurkers at h' . (Lemma 12)
 - If a lurked object is clinchable at h' , then there is no passing action at h' (that is, h' is a terminating history; see footnote 42). (Lemma 14)
- At some history $h^k = (h_\emptyset^k, a^*, \dots, a^*)$, an agent $i \notin \mathcal{L}(h^k)$ takes a clinching action $a_x \in A(h^k)$ (it is possible that $h^k = h_\emptyset^k$, in which case the first agent called to play in the round clinches an object, and there are no passing actions). The history (h^k, a_x) begins the clinching phase.

Clinching phase:

- In the clinching phase, following the clinching of a lurked object x , all lurkers at h^k who lurk objects strictly older than x clinch their lurked object (see footnote 37). These agents may or may not be called to move, and if any of them are called to move, they move in order from oldest to youngest lurker, have only clinching actions, and never move again. Following the last clinching action of these agents, the next agent to move is the agent who lurked x at h^k (notice that some other agent is the one who clinches x), and she is given at least a clinching action for each lurked object strictly younger than x and each unlurked object. There is no passing action. (Lemma 16)
- The game continues in this manner: each time a lurked object x' is clinched, all lurkers who lurk objects strictly older than x' clinch their lurked object with or without being called to move. After these agents (if any) are called to move, the next agent

case we assume that $x \notin C_{j'}^{\leq}(h)$, all of the objects in $C_{j'}^{\leq}(h)$ remain unassigned, and thus may be offered to j' . See Lemma 18 in Appendix C for details.

⁴²Following BG, we call such h' the *terminating history* of the round; formally, the terminating history of round k is the maximal superhistory of the form $h' = (h_\emptyset^k, a^*, \dots, a^*)$. Note that the terminating history is not a terminal history, nor is it necessarily the conclusion of the current round (which will be defined recursively below); rather, it is the “termination” of the passing path. Depending on the preferences, the path through the mechanism may or may not reach the terminating history of a round. See Appendix C for details.

called to move is the agent who lurked x' , and she is given at least a clinching action for each lurked object strictly younger than x' and each unlurked object.

- At some point in this sequence of moves, some agent—either an agent who was a lurker at h^k , or possibly i , the non-lurker who moved at h^k to initiate the clinching phase—clinches an unlurked object $\hat{x} \in \bar{\mathcal{X}}^{\mathcal{L}}(h^k)$. All remaining lurkers clinch their lurked object with or without being called to move. If any of these lurkers are called to move, let $\hat{h}$ be the history that ends with the action of the last of these agents; otherwise, let $\hat{h}$ be the history that ends with the clinching of $\hat{x}$. (Lemma 16)
- At $\hat{h}$, there might be at most one remaining active agent j who has not yet clinched an object; if such a j exists, they were active at h^k , but did not move at h^k nor were they a lurker h^k .⁴³ If such a j exists and $\hat{x} \in C_j^{\subseteq}(h^k)$, then agent j is called to move at $\hat{h}$ and is offered at least a clinching action for each object that is still possible for her. There is no passing action. (Lemma 17)

The history $\hat{h}$ is the beginning of round $k + 1$ (and thus the end of round k), except if there is an agent j as described in the last bullet point, and $\hat{x} \in C_j^{\subseteq}(h^k)$; in this latter case, round $k + 1$ begins following the clinching action of this j . Thus, in the former case, the initiating history for round $k + 1$ is $h_{\emptyset}^{k+1} = \hat{h}$, and in the latter case, it is $h_{\emptyset}^{k+1} = (\hat{h}, a_j)$, where a_j is the clinching action of agent j at $\hat{h}$.

If all agents active at h^k have clinched an object by $h_{\emptyset}^{k+1}$, then some previously inactive agent i' moves at $h_{\emptyset}^{k+1}$ (unless all agents have clinched an object, in which case the game ends). If there remains an agent j who was active at h^k and has not clinched an object at $h_{\emptyset}^{k+1}$ —which can only occur when there exists an active non-lurker j at h^k who does not move at h^k and $\hat{x} \notin C_j^{\subseteq}(h^k)$ —then Lemma 18 implies that j moves at $h_{\emptyset}^{k+1}$ to begin the next round, and is offered for clinching some set of objects $C_j(h_{\emptyset}^{k+1}) \supseteq C_j^{\subseteq}(h_{\emptyset}^{k+1})$.⁴⁴ At $h_{\emptyset}^{k+1}$, agent j may be offered to clinch additional objects beyond those in $C_j^{\subseteq}(h_{\emptyset}^{k+1})$, and may or may not have a passing action.

In either case, the continuation game starting at $h_{\emptyset}^{k+1}$ is another Pareto efficient and pruned millipede game on the remaining unmatched agents and objects.⁴⁵ This continuation game starting from $h_{\emptyset}^{k+1}$ also satisfies properties 1–5, and so continues to have the same passing-and-clinching phase structure.

Step 4: Coding Algorithm

By Lemma 2, we may assume that the mechanism (Γ, S) in Proposition 2 is a millipede mechanism satisfying properties 1–5. At the core of the remainder of the proof of Proposition 2 is the construction of a bijection between role assignment functions for the permuted millipede mechanisms that make up the symmetrization of (Γ, S) and serial dictatorship orderings such that the outcomes of the permuted millipede and permuted serial dictatorship are exactly the same. More formally, let Ord denote the set of total linear orders over the set of agents $\mathcal{N}$. Random Priority draws an agent ordering uniformly

⁴³Such agent j also does not move at any h' such that $h^k \subseteq h' \subsetneq \hat{h}$.

⁴⁴Notice that j does not move between h^k and $h_{\emptyset}^{k+1}$, so $C_j^{\subseteq}(h_{\emptyset}^{k+1}) = C_j^{\subseteq}(h^k)$.

⁴⁵In particular, when restricted to the objects that are not yet clinched at $h_{\emptyset}^{k+1}$, all possible ranking types are present at $h_{\emptyset}^{k+1}$ for all remaining agents. For those agents who have never been called to move, this is obvious, as they have not yet revealed any preference information. For the agent j who might be carried over from the previous round, it follows because each type that ranks $\hat{x}$ first (over the set of all objects $\mathcal{X}$) reaches $h_{\emptyset}^{k+1}$, and $\hat{x}$ is no longer available at $h_{\emptyset}^{k+1}$; cf. Lemma 18.

at random from Ord , and thus (for a fixed preference profile) the probability of any particular allocation μ is just the number of agent orderings such that a serial dictatorship under such an ordering results in μ , divided by $N!$, the total number of possible agent orderings. Similarly, in the symmetrization of (Γ, S) , the probability of μ is the number of role assignment functions $\sigma \in \Sigma$ such that the permuted mechanism $(\Gamma_\sigma, S_\sigma)$ results in μ divided by $N!$. Thus, if we can find a bijection $f : \Sigma \rightarrow Ord$ such that, for every $\sigma \in \Sigma$, the permuted mechanism $(\Gamma_\sigma, S_\sigma)$ results in the same allocation as a serial dictatorship under agent ordering $f_\sigma(1), \dots, f_\sigma(N)$ —where $f_\sigma(j)$ denotes the j th ranked agent under the agent ordering f_σ —the distribution over allocations in the symmetrized millipede will have been shown to be the same as the distribution over allocations in Random Priority, which will prove Proposition 2 (and hence, also Theorem 1).

The rest of the proof is devoted to constructing the necessary bijection f . In step 4 here, we introduce a coding algorithm that takes a continuation game under a role assignment function Γ_σ and maps (or “codes”) it to a partial ordering of the agents, denoted $\succ$. This partial ordering may include ties, and steps 5 and 6 below show how to take these partial orderings and break ties to obtain the full bijection $f : \Sigma \rightarrow Ord$.

The intuitive idea behind constructing $\succ$ is as follows. Fixing a preference profile and beginning with round 1 of the mechanism, we find the first agent to clinch some object x after a (possibly empty) series of passes at some history h^1 . This initiates the clinching phase of round 1, in which all but at most one of the agents who are active at h^1 clinch their objects, as described in step 3 of the proof. We create $\succ$ by ordering agents who receive lurked objects in order of the “age” of the object they received, that is, the first agent in the ordering is the agent who receives the object that became lurked first, the second is the agent who received the object that became lurked second, and so forth (note that this is different from ordering lurkers by their age, as a lurker may end up receiving a different object than the one she lurked).

After ordering the agents who receive lurked objects, there are at most two active agents who have yet to be coded, one of whom has clinched an unlurked object, say $\hat{x}$. If $\hat{x}$ was previously offered to the remaining active agent (if such an agent exists), then we add both of these agents to the order without distinguishing between them, that is, these two agents tie.⁴⁶ If $\hat{x}$ was not previously offered to the other remaining active agent (or no such agent exists), then we only add to the ordering the agent who clinched $\hat{x}$. The other active agent (if such an agent exists) will be added in a later step triggered by a later clinching; at the beginning of the next round, this agent is still active with the carried over “endowment” of previously clinchable objects, $C_j^\subseteq(h)$ (see Remark 2). After clearing this first segment of agents, we repeat this procedure starting from the initiating history of the next round.

To illustrate, consider Figure 3, which reproduces the game from Section 4 under the role assignment function $\sigma''(r_1) = i_3$, $\sigma''(r_2) = i_2$, and $\sigma''(r_3) = i_1$. The path of play of the game under this role assignment and the given preference profile (which is restated in the caption of the figure) is shown by the dashed lines. Agent i_3 moves first, and her top choice is z . She is offered to clinch only x and y at her first move, while z is possible later in the game if she passes, and so her obviously dominant strategy is to pass. Upon passing, i_3 has now been offered all objects that are possible for her in the game except for one (object z), and thus i_3 becomes a lurker, and z becomes a lurked object. We follow the dashed line until we find the first agent to clinch, which in this case is agent i_1 , who

⁴⁶Breaking this tie is a key subtlety in the construction of the bijection from role assignments to orderings of agents. We show how to do so in step 6.

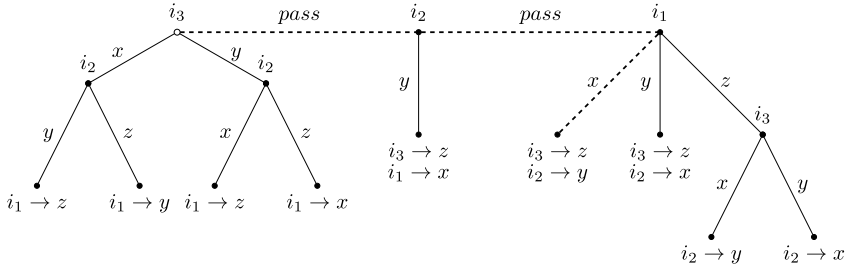


FIGURE 3.—The example from Section 4 under the role assignment $\sigma''(r_1) = i_3$, $\sigma''(r_2) = i_2$, and $\sigma''(r_3) = i_1$ and preferences $>_{i_1}: x, y, z$, $>_{i_2}: x, y, z$, $>_{i_3}: z, y, x$. The dashed lines show the path of play.

clinches object x . This triggers the ordering of the currently active agents—which in this case is all of the agents—and orders them by first ordering agents who receive lurked objects according to the age of the lurked object they receive. In this case, both z and x are lurked (in this order), and so the corresponding serial dictatorship is $f_{\sigma''}(1) = i_3$, $f_{\sigma''}(2) = i_1$, and $f_{\sigma''}(3) = i_2$.

We now present the formal definition of the coding algorithm just described.

Coding Algorithm. Consider a permuted mechanism $(\Gamma_\sigma, S_\sigma)$, and fix a preference profile $>_{\mathcal{N}}$. Consider the game path from the root node $h_\emptyset$ to a terminal node $\bar{h}$ when agents follow the strategy profile $S_\sigma(>_{\mathcal{N}})$. Each step k of the algorithm below produces a partial ordering $\succ^k$ on the set of agents who are processed in step k . At the end of the final step K , we concatenate the K components to produce $\succ$, the final coding on the set of all agents $\mathcal{N}$.

Step 1 Let $h_\emptyset^1 = h_\emptyset$ and $h^1 = (h_\emptyset^1, a^*, \dots, a^*)$ be the history that initiates the first clinching phase of the game (it is possible that $h^1 = h_\emptyset^1$, when the first agent to move takes a clinching action). Let x_1 be the object clinched by agent i_1 at h^1 .⁴⁷ Let $\mathcal{L}(h^1) = \{\ell_1, \dots, \ell_{\lambda(h^1)}\}$ be the set of lurkers, and $\mathcal{X}^{\mathcal{L}}(h^1) = \{x_1, \dots, x_{\lambda(h^1)}\}$ be the set of lurked objects at h^1 , where x_k is the k th object to become lurked and ℓ_k the lurker of this object; if these sets are empty, skip directly to step 1.2 below.

1. For $x_k \in \mathcal{X}^{\mathcal{L}}(h^1)$, let i_{x_k} be the agent who receives x_k at $\bar{h}$.⁴⁸
2. Let $j \in \mathcal{L}(h^1) \cup \{i^1\}$ be the unique agent that is not one of the agents $i_{x_1}, \dots, i_{x_{\lambda(h^1)}}$ from step 1.1. Because we restricted attention to millipedes satisfying properties 1–5 above, j receives an unlurked object $y \in \bar{\mathcal{X}}^{\mathcal{L}}(h^1)$ and there may be at most one active agent $j' \in \mathcal{A}(h^1) \setminus (\mathcal{L}(h^1) \cup \{i^1\})$.
 - (a) If such a j' exists and $y \in C_{j'}^{\subseteq}(h^1)$, then define $\tilde{\succ}^1$ as

$$i_{x_1} \tilde{\succ}^1 i_{x_2} \tilde{\succ}^1 \dots \tilde{\succ}^1 i_{x_{\lambda(h^1)}} \tilde{\succ}^1 \{j, j'\}.$$

⁴⁷That is, $i_{h^1} = i^1$, and i^1 selects a clinching action $a_{x^1} \in \mathcal{A}(h^1)$ that clinches x^1 . By Lemma 6, $i^1 \notin \mathcal{L}(h^1)$. Notice the difference between superscript in x^1 , which refers to the step of the algorithm, and the subscripts in lurked objects, which refer to the order in which they become lurked. In the notation for lurkers $\ell_k^{h^1}$ and lurked objects $x_k^{h^1}$ we suppress the history superscript.

⁴⁸Note that i_{x_k} is not necessarily the agent who lurks x_k at h^1 .

(b) Otherwise, define $\succsim^1$ as

$$i_{x_1} \succsim^1 i_{x_2} \succsim^1 \dots \succsim^1 i_{x_{\lambda(h^1)}} \succsim^1 j.$$

In particular, if j' exists and $y \notin C_{j'}^\subseteq(h^1)$, then we do not yet order agent j' .

Step k Let $h_\emptyset^k$ be the initiating history that begins round k of the mechanism (as described in Remark 2), and let $h^k = (h_\emptyset^k, a^*, \dots, a^*)$ be the history that initiates the round k clinching phase (as in step 1, it is possible that $h^k = h_\emptyset^k$). Let x^k be the object clinched by i^k at history h^k . Let $\mathcal{L}(h^k) = \{\ell_1, \dots, \ell_{\lambda(h^k)}\}$ be the set of lurkers, and $\mathcal{X}^\mathcal{L}(h^k) = \{x_1, \dots, x_{\lambda(h^k)}\}$ be the set of lurked objects, and carry out a procedure analogous to that from step 1 to produce the step k order $\succsim^k$.

This produces a collection of codings $(\succsim^1, \dots, \succsim^K)$, where each $\succsim^k$ is a partial order on the agents processed in step k . We then create the final $\succ$ in the natural way: for any two agents i, j who were processed in the same step k , $i \succ j$ if and only if $i \succsim^k j$. For any two agents i, j processed in different steps k and k' respectively, where $k < k'$, we order $i \succ j$.

The output of the coding algorithm is a partial order, $\succ$, on $\mathcal{N}$, the set of agents. If $i \succ j$, we say that i **precedes** j . If there are two agents i and j such that $i \not\succ j$ and $j \not\succ i$, then we say i and j **tie** under $\succ$. We also use the notation $i \succ \{j, k\} \succ \ell$ to denote that i precedes j and k , the latter two agents tie, and in turn these two agents precede ℓ . Note that by construction, all ties are of size at most 2, and agents can only tie if they are processed in the same step of the algorithm.

REMARK 3: Different preference profiles $\succ_{\mathcal{N}}$ may lead to different codings. For a given $\succ_{\mathcal{N}}$, the coding algorithm divides the game path from the root to the terminal node into a series of K steps. Each step of the coding algorithm corresponds precisely to a round of the mechanism (as described in Remark 2), and the agents coded in step k are precisely those agents who clinched an object during the round k clinching phase of the mechanism.⁴⁹ In step $k + 1$ of the coding algorithm, we start at the initiating history $h_\emptyset^{k+1}$, and find the history $h^{k+1} = (h_\emptyset^{k+1}, a^*, \dots, a^*)$ that corresponds to the first clinching action taken by an agent in round $k + 1$ of the mechanism. The step $k + 1$ coding is the equivalent to the initial coding on the smaller Pareto efficient millipede continuation game that begins at history $h_\emptyset^{k+1}$.

Each role assignment function σ induces a permuted mechanism $(\Gamma_\sigma, S_\sigma)$, and each permuted mechanism has an associated coding $\succ_\sigma$ obtained via applying the coding algorithm to the mechanism $(\Gamma_\sigma, S_\sigma)$. This results in a collection of $N!$ codings $(\succ_\sigma)_{\sigma \in \Sigma}$. Codings do not map directly to serial dictatorship orderings, because some agents may tie. In the remainder of the proof, we show that (i) no matter how these ties are broken, the resulting serial dictatorship results in the same allocation as the original game $(\Gamma_\sigma, S_\sigma)$ (step 5) and (ii) it is possible to break ties across all of the $N!$ codings in such a way that the resulting mapping from permuted games to serial dictatorship orderings is a bijection (step 6).

⁴⁹ At the end of coding step k , there may be one agent, say j' , who was active during round k of the mechanism, and was not coded in step k . When this occurs, at the initiating history for round $k + 1$, $h_\emptyset^{k+1}$, agent j' is called to move, and is offered to clinch at least each object that she has been offered to clinch previously in the game (and might have other moves), as explained in Remark 2.

Step 5: Same Allocations

Take a role assignment function σ and the resulting coding $\succ_\sigma$. We say that a total ordering of the agents f_σ is **consistent** with $\succ_\sigma$ if, for all j, j' : $j \succ_\sigma j'$ implies $f_\sigma^{-1}(j) < f_\sigma^{-1}(j')$. In other words, given some coding $\succ_\sigma$, total order f_σ is consistent if there is some possible way to break the ties in $\succ_\sigma$ that delivers f_σ . We further say that f_σ is **consistent with $\succ_\sigma$ on an initial segment till an agent i** if, for all j, j' that either precede i or tie with i , if $j \succ_\sigma j'$, then $f_\sigma^{-1}(j) < f_\sigma^{-1}(j')$.

LEMMA 3: *For any agent i and any total order f_σ consistent with $\succ_\sigma$ on an initial segment till i , the allocation of agents who precede or tie with i under the serial dictatorship with agent ordering f_σ is the same as their allocation in Γ_σ . In particular, given two games Γ_A and Γ_B played under role assignment functions σ_A and σ_B , respectively, if $\succ_A = \succ_B$, then Γ_A and Γ_B end with the same final allocations to all agents.*

We prove this lemma in Appendix C.2. Given $\succ_\sigma$, any way of breaking the ties (if any ties exist) between agents produces a total order f_σ that is consistent with $\succ_\sigma$. Thus, by Lemma 3, no matter how ties are broken, the mechanism $(\Gamma_\sigma, S_\sigma)$ ends with the same allocation as the serial dictatorship with agent ordering f_σ .

Step 6: Bijectivity

Finally, we show that it is possible to break the ties in the codings produced by the coding algorithm, $(\succ_\sigma)_{\sigma \in \Sigma}$, in such a way as to produce a mapping $f : \Sigma \rightarrow \text{Ord}$ that is a bijection. We prove bijectivity using two lemmas—Lemmas 4 and 5—on the properties of the partial orders produced by the coding algorithm applied to games with different role assignments. The proofs of these lemmas can be found in Appendix C.2.

Let h_A^k be the history that initiates step k of the coding algorithm when it is applied to game Γ_A . For instance, $h_A^1 = (h_\emptyset, a^*, \dots, a^*)$ is a history following a (possibly empty) sequence of passes such that agent $i_{h_A^1}$ moves at h_A^1 and is the first agent to clinch in the game. This induces a chain of assignments of the agents in $\mathcal{L}(h_A^1) \cup \{i_{h_A^1}\}$, plus possibly one other active non-lurker at h_A^1 , as given in the description of millipede mechanisms with lurkers. History $h_A^2 \supsetneq h_A^1$ is then the next time along the game path that an agent who was not ordered in step 1 of the coding algorithm clinches an object, etc. Define h_B^k analogously, and let K_A and K_B be the total number of steps in the coding algorithm when applied to games Γ_A and Γ_B , respectively.

LEMMA 4: *Let σ_A and σ_B be two role assignment functions, and Γ_A and Γ_B their associated games. Let $\succ_A^k$ be the initial segment of $\succ_A$ consisting of agents ordered up to and including step k of the coding algorithm in game Γ_A . If ordering $\succ_A^k$ equals to an initial segment of $\succ_B$, then $h_A^{k'} = h_B^{k'}$ for all $k' = 1, \dots, k$ and $\sigma_A^{-1}(i) = \sigma_B^{-1}(i)$ for all agents i who are coded up to step k . In particular, if $\succ_A = \succ_B$, then $h_A^k = h_B^k$ for all k , $K_A = K_B$, and $\sigma_A^{-1}(i) = \sigma_B^{-1}(i)$ for all $i \in \mathcal{N}$.*

The proof of Lemma 4 can be found in Appendix C.2. The lemma shows that the mapping from role assignments to codings is injective. Each coding is a partial ordering and what we are ultimately after is a mapping from role assignments to strict total orderings of the agents. We generate these strict orderings by breaking ties in partial orderings. In effect, what remains to be shown is that it is possible to break the ties in all codings in a

way that maps each partial order to a unique strict order. Lemma 5 provides the key tool needed to do this.

We write $j_1 \cdots j_P \succ i \succ j \succ \cdots$ when $\succ$ ranks $j_1, \dots, j_P$ first, possibly with ties; ranks i immediately (and strictly) after, and then ranks j immediately (and strictly) after i . We write $j_1 \cdots j_P \succ i \succ \{j, k\} \cdots$ when $\succ$ ranks $j_1, \dots, j_P$ first, possibly with ties, and then ranks the tie $\{j, k\}$ immediately after. We write $j_1 \cdots j_P \succ i \succ j \cdots$ to denote the case in which either of the two previous possibilities may hold (i.e., j may or may not tie with some other agent k).

Before stating the next lemma, to understand the issue better, we discuss a small example. Say that there exists some role assignment function such that the coding to partial orders is $\{i, k_1\} \succ_1 \cdots$. To create a strict ordering of the agents, we must (somehow) break the tie between i and k_1 . By Lemma 3, either way of breaking this tie will produce a serial dictatorship that gives the same allocations as Γ_1 , and so from this perspective, it does not matter how the tie is broken. While Lemma 4 shows that the mapping of role assignments to partial orders is injective, we also need to ensure that after any tie-breaking, the mapping to strict total orders continues to be injective, which means that no other coding is mapped to the same strict total order after the tiebreak. In particular, there might be another coding $k_1 \succ_2 i \succ_2 \cdots$. To preserve injectivity, we then break the tie in $\succ_1$ so that $i \succ_1 k_1 \succ_1 \cdots$ (because if we broke the tie such that $k_1 \succ_1 i \succ_1 \cdots$, both $\succ_1$ and $\succ_2$ could map to the same strict total order, violating injectivity). Can we do so without a conflict with some other coding $\succ'$ that also begins with $i \succ' k_1 \succ' \cdots$? Lemma 5 below is at the core of our argument that we can. Indeed, if there is some $\succ_3$ such that $i \succ_3 k_1 \cdots$, then Lemma 5 implies that k_1 must tie with some other agent k'_2 . Therefore, in the coding $\succ_3$, we can break the tie so that $i \succ_3 k'_2 \succ_3 k_1 \succ_3 \cdots$, and in so doing, ensure that there are no injectivity conflicts with the tiebreak of $\succ_1$. We must also make sure that after breaking the tie in $\succ_3$, there is also no conflict with any other $\succ_4$ such that $i \succ_4 k'_2 \succ_4 k_1 \succ_4 \cdots$, and so forth. Lemma 5 allows us to do so.

To formulate the lemma, we introduce the notion of an arm. Given a coding $\succ_1$ in which two agents i and k_1 tie, an **arm** of length $n + 1 \geq 1$ is a sequence of $n + 1$ role assignments $\alpha = (\sigma_1, \sigma_2, \dots, \sigma_n, \sigma_{n+1})$ such that the resulting codings have the following structure:

$$\begin{aligned} \text{Arm } \alpha: \quad & j_1 \cdots j_P \succ_1 \{i, k_1\} \succ_1 \cdots \\ & j_1 \cdots j_P \succ_2 k_1 \succ_2 \{i, k_2\} \succ_2 \cdots \\ & j_1 \cdots j_P \succ_3 k_1 \succ_3 k_2 \succ_3 \{i, k_3\} \succ_3 \cdots \\ & \vdots \\ & j_1 \cdots j_P \succ_n k_1 \succ_n k_2 \succ_n k_3 \succ_n \cdots \succ_n k_{n-1} \succ_n \{i, k_n\} \succ_n \cdots \\ & j_1 \cdots j_P \succ_{n+1} k_1 \succ_{n+1} k_2 \succ_{n+1} k_3 \succ_{n+1} \cdots \succ_{n+1} k_{n-1} \succ_{n+1} k_n \succ_{n+1} i \cdots \end{aligned}$$

If $n = 0$, then the arm α consists of just one coding $j_1 \cdots j_P \succ_1 \{i, k_1\} \succ_1 \cdots$. Note that an arm α satisfies the following properties: (i) the partial orderings of all codings coincide on all agents ranked above k_1 (represented by $j_1 \cdots j_P$, and we allow that there may be ties among some of the j_p 's); (ii) the first coding in the arm ranks the tie $\{i, k_1\}$ strictly below $j_1 \cdots j_P$; (iii) there is a set of agents $k_2, \dots, k_n$ such that the n th element of the arm ranks as shown above. In particular, in coding $\succ_n$, agent i ties with k_n , while in $\succ_{n+1}$, agent i may or may not tie with some other agent.

If there exists a coding $\succ_1$ with a tie $\{i, k_1\}$ and an arm α of the above form in which i does not tie with another agent under $\succ_{n+1}$, then, in order to preserve injectivity, we break

the tie in $\succ_1$ such that $i \succ_1 k_1$; in this case, we say that α **breaks the tie** $\{i, k_1\}$ in $\succ_1$ in favor of i .

LEMMA 5: Suppose that for positive integers $n, m \geq 0$, there are two arms $\alpha = \{\sigma_1, \sigma_2, \dots, \sigma_n, \sigma_{n+1}\}$ and $\alpha' = \{\sigma'_1, \sigma'_2, \sigma'_3, \dots, \sigma'_m, \sigma'_{m+1}\}$ such that $\sigma_1 = \sigma'_1$ and:

$$\begin{aligned}
 \text{Arm } \alpha: \quad & j_1 \cdots j_P \succ_1 \{i, k_1\} \succ_1 \cdots \\
 & j_1 \cdots j_P \succ_2 k_1 \succ_2 \{i, k_2\} \succ_2 \cdots \\
 & j_1 \cdots j_P \succ_3 k_1 \succ_3 k_2 \succ_3 \{i, k_3\} \succ_3 \cdots \\
 & \vdots \\
 & j_1 \cdots j_P \succ_n k_1 \succ_n k_2 \succ_n k_3 \succ_n \cdots \succ_n k_{n-1} \succ_n \{i, k_n\} \succ_n \cdots \\
 & j_1 \cdots j_P \succ_{n+1} k_1 \succ_{n+1} k_2 \succ_{n+1} k_3 \succ_{n+1} \cdots \succ_{n+1} k_{n-1} \succ_{n+1} k_n \succ_{n+1} i \cdots, \\
 \text{Arm } \alpha': \quad & j_1 \cdots j_P \succ'_1 \{i, k_1\} \succ'_1 \cdots \\
 & j_1 \cdots j_P \succ'_2 i \succ'_2 \{k_1, k'_2\} \succ'_2 \cdots \\
 & j_1 \cdots j_P \succ'_3 i \succ'_3 k'_2 \succ'_3 \{k_1, k'_3\} \succ'_3 \cdots \\
 & \vdots \\
 & j_1 \cdots j_P \succ'_m i \succ'_m k'_2 \succ'_m k'_3 \succ'_m \cdots \succ'_m k'_{m-1} \succ'_m \{k_1, k'_m\} \succ'_m \cdots \\
 & j_1 \cdots j_P \succ'_{m+1} i \succ'_{m+1} k'_2 \succ'_{m+1} k'_3 \succ'_{m+1} \cdots \succ'_{m+1} k'_{m-1} \succ'_{m+1} k'_m \succ'_{m+1} k_1 \cdots,
 \end{aligned}$$

where the partial order on $j_1 \cdots j_P$ is the same in all above codings. Then, one of the following must hold:

- (I) in $\succ_{n+1}$, agent i ties with some agent k_{n+1} ; or
- (II) in $\succ'_{m+1}$, agent k_1 ties with some agent k'_{m+1} .

If either n or m is 0, the lemma is a tautology. The proof shows that it continues to hold for arbitrary $n, m \geq 1$.

We call such α and α' as defined in the lemma **dual arms**. They have the following properties:

- Each arm starts with the same role assignment and codings, that is, $\sigma_1 = \sigma'_1$ and $\succ_1 = \succ'_1$, in which i and k_1 tie.
- In arm α , every subsequent coding ranks k_1 strictly ahead of all other agents (besides the j_p 's), while in α' , every subsequent coding ranks i ahead of all other agents (besides the j_p 's).
- Within arm α , the only difference from ℓ to $\ell + 1$ is that the agent k_ℓ who tied with i in $\succ_\ell$ is now ranked strictly above i , with i now tied with a different agent, $k_{\ell+1}$ (except for $\succ_{n+1}$, in which case i is ranked next, but may or may not tie with another agent). A similar remark applies to α' .
- Across the two arms, it is possible that some or all of the agents $k_2, \dots, k_n$ are the same as the agents $k'_2, \dots, k'_m$, though it is not necessarily assumed.
- We do not require $m = n$.
- In arm α , any agent $\ell \in \{j_1, \dots, j_P, i, k_1\}$ receives the same object for all role assignments $\sigma_1, \dots, \sigma_{n+1}$ of the arm. Indeed, there is an ordering of agents $j_1, \dots, j_P, k_1, i$

that is consistent with both codings $\succ_1$ and $\succ_2$ and hence Lemma 3 implies that each $\ell \in \{j_1, \dots, j_P, i, k_1\}$ receives the same objects under σ_1 and σ_2 . Similarly, there is an ordering $j_1, \dots, j_P, k_1, k_2, i$ that is consistent with both $\succ_2$ and $\succ_3$, and so by Lemma 3 again, each $\ell \in \{j_1, \dots, j_P, i, k_1\}$ receives the same object under σ_2 and σ_3 (and hence, this is also the same as the object received under σ_1). Continuing in this way, we conclude that all agents $j_1, \dots, j_P, i, k_1$ receive the same object under all role assignments in the arm. A similar argument shows that for $n' \in \{2, \dots, n\}$, agent $k_{n'}$ receives the same object for all role assignments $\sigma_{n'}, \sigma_{n'+1}, \dots, \sigma_{n+1}$.⁵⁰ An analogous observation holds for arm α' . Notice also that both arm α and α' begin with the same role assignment, and so this further implies that the agents $j_1, \dots, j_P, i, k_1$ receive the same objects for all role assignments in both arms.

The proof of Lemma 5 can be found in Appendix C.2. The upshot of the lemma is that, given some coding $\succ_1$ with a tie $\{i, k_1\}$ if there exists an arm α that breaks the tie in favor of i , then, in the final coding of any dual arm α' , agent k_1 must tie, and thus breaking the tie in $\succ_1$ in favor of i will not lead to a conflict with another coding that will break injectivity.

More formally, let Ψ be the initial mapping from role assignments σ to partial orderings $\succ_\sigma$ generated by the coding algorithm from step 4 of the proof. By Lemma 4, Ψ is injective. We use Lemma 5 to break ties to create from each $\succ_\sigma$ a consistent total order f_σ in a way that preserves the injectivity by iteratively executing the following tie-breaking step. We break ties one at a time, and after breaking a tie, we update the mapping Ψ so that $\Psi(\sigma)$ is the previous partial ordering except for the now-broken former tie. Importantly, the above concept of an arm is applicable to any collection of partial orderings, and we show below that injectivity and the claim of Lemma 5 continue to hold after each tie is broken.

Tie-Breaking Step. Let $P \in \{1, \dots\}$ be the smallest integer such that there is some role assignment σ and agents $j_1, \dots, j_{P-1}, i, k_1$ such that the initial segment of the partial ordering $\Psi(\sigma)$ is $j_1 \succ_\sigma \dots \succ_\sigma j_{P-1} \succ_\sigma \{i, k_1\}$.⁵¹ If such an integer P does not exist, then all ties are already broken and the tie-breaking procedure terminates. There are two cases:

(a) If there exists such σ for which there is an arm α breaking the tie $\{i, k_1\}$ in position P in favor of one of the agents, say i , then we choose one such σ and break this tie accordingly. By Lemma 5, this is well-defined, in the sense that if there is an arm that breaks the tie in favor of i , then there is no arm α' that breaks the tie in favor of k_1 .

(b) Else, we choose an arbitrary σ with a tie in position P and we break it arbitrarily.

After executing a single tie-break by either step (a) or (b) above, we update the mapping Ψ so that it is the same as before, except with the now-broken tie in $\Psi(\sigma)$; without loss of generality, we denote this broken tie in favor of i , that is, $i \succ_\sigma k_1$.

For the updated Ψ and the resulting collection of partial orderings, we show the following: (i) Ψ continues to be injective and (ii) the conclusion of Lemma 5 continues to hold.

For (i), towards a contradiction, assume that injectivity fails after we broke the tie at role assignment σ . Since injectivity held before the tie-break, and the only tie broken was in σ , we must have that the updated $\Psi(\sigma)$ is the same partial ordering as $\Psi(\sigma')$ for some $\sigma' \neq \sigma$. Thus, before the tie-break, $\Psi(\sigma')$ must have been a partial ordering

⁵⁰Notice that this does not apply for role assignments $\sigma_{n''}$ where $n'' < n'$, because in such role assignments $k_{n'}$ is ordered below i .

⁵¹Note that here we index the j agents from $1, \dots, P-1$ (rather than P for most of the rest of the proof) so that we can refer to a tie in the P th (rather than $(P+1)$ th) position. Of course, this indexing is arbitrary.

that starts with $j_1 \succ_{\sigma'} \cdots \succ_{\sigma'} j_{P-1} \succ_{\sigma'} i \succ_{\sigma'} k_1 \succ_{\sigma'} \cdots$. This implies that before the tie-break, $\alpha' = (\sigma, \sigma')$ is an arm that breaks the tie in σ in favor of k_1 , which contradicts Lemma 5.⁵²

For (ii), suppose that after the tie-break in σ , there are now two dual arms α and α' that each begin with $j'_1, \dots, j'_{P-1} \succ_1 \{i', k'_1\}$ for some role assignment σ_1 and that have the structure as in the statement of Lemma 5. As in the lemma, let the length of arm α be $n + 1$, and the length of arm α' be $m + 1$. Further suppose that, in violation of the claim of Lemma 5, partial ordering $\succ_{n+1}$ in arm α ranks i' strictly, and partial ordering $\succ'_{m+1}$ in arm α' ranks k_1 strictly; note in particular that this implies that $n, m \geq 1$.

By the order in which we break the ties, all ties before P have been already broken, and hence $P' \geq P$. Because the claim of the lemma was true before we broke the tie $\{i, k_1\}$ at role assignment σ , it must then be that after the tie-break, $j_1 \succ_{\sigma} \cdots \succ_{\sigma} j_{P-1} \succ_{\sigma} i \succ_{\sigma} k_1 \succ_{\sigma}$ is part of at least one of these arms; without loss of generality, assume that this is arm α , and so $\succ_{\ell} = \succ_{\sigma}$ for some $\ell \leq n + 1$ in arm α (an analogous argument applies to the case that the relevant arm is α'). This implies that $j'_1 = j_1, \dots, j'_{P-1} = j_{P-1}$. Further, we know that i and k_1 are ranked strictly in positions P and $P + 1$, which implies $j'_P = i, j'_{P+1} = k_1$, and $P' \geq P + 2$. Next, notice that before the tie-break, there cannot be any other role assignment $\sigma'' \neq \sigma$ whose corresponding partial ordering begins with $j_1 \succ_{\sigma''} \cdots \succ_{\sigma''} j_{P-1} \succ_{\sigma''} i \succ_{\sigma''} k_1 \succ_{\sigma''}$.⁵³ Because the only tie that was broken was in the partial ordering for role assignment σ , the same holds true after the tie-break as well. This implies that the arm α must be a trivial arm ($n = 0$), because if $n \geq 1$, then, by the structure of an arm and the fact that $P' \geq P + 2$, there are at least two partial orderings that begin by ranking $j_1, \dots, j_{P-1}, i, k_1$ strictly and in this order, in contradiction with the previous sentence. However, $n = 0$ contradicts the previously reached conclusion that $n \geq 1$. This contradiction shows that Lemma 5 continues to hold for the updated mapping Ψ .

We continue executing the above tie-breaking step until we reach a mapping Ψ from role assignments to complete orderings (with no ties remaining). Following each individual tie-break, we can repeat the argument just made to conclude that injectivity is preserved, and thus the final Ψ is injective. As the resulting total orderings are created by breaking ties in the original codings, the complete orderings are consistent with the original codings. Hence, we created an injective mapping from permutations to total orderings that are consistent with codings. In this way, we obtain an injection from role assignments σ to serial dictatorships with orders f_{σ} . Because in this injection the domain of role assignments σ and the range of serial dictatorship orderings f_{σ} are finite and have equal size, this injection is a bijection.

Recap

To recap, we have shown the following:

1. Every Pareto efficient, obviously strategy-proof mechanism (Γ, S) is equivalent to a (perfect-information) millipede mechanism satisfying properties 1–5 in which Nature moves once (if at all) as the first mover (Lemma 2).

⁵²For further detail, notice that if the tie-break in σ was broken via (a) above, then we have two dual arms, α and α' , one of which breaks the tie in favor of i and the other that breaks the tie in favor of k_1 , which is a contradiction to Lemma 5. If the tie was broken via (b) above, then there must not have been any tie for which there was an arm that broke the tie in favor of one of the agents, which again is a contradiction.

⁵³This follows via a similar argument in the proof of claim (i) in the paragraph above: if such a σ'' existed, then, before the tie-break, there was an arm $\alpha'' = (\sigma, \sigma'')$ that breaks the tie in σ in favor of k_1 , and we reach a contradiction via the same argument as in footnote 52.

2. For any millipede mechanism satisfying properties 1–5, there is a bijection f between role assignment functions and serial dictatorship orderings such that the final allocation of the permuted mechanism $(\Gamma_\sigma, S_\sigma)$ results in the same final allocation as a serial dictatorship using the agent ordering f_σ (Lemmas 3, 4, and 5).
3. Point (2) implies that the symmetrization of (Γ, S) is equivalent to Random Priority (see the argument in the first paragraph of step 4).
4. Since the symmetrization of every obviously strategy-proof, Pareto efficient, and deterministic perfect-information mechanism (Γ, S) is equivalent to Random Priority, by Lemma 1 every symmetric, obviously strategy-proof, and Pareto efficient mechanism is equivalent to Random Priority.

APPENDIX C: PROOFS OF KEY LEMMAS 2–5

C.1. Proof of Lemma 2

As noticed by Li (2017), without loss of generality we can assume that the mechanism (Γ, S) is pruned. Properties 1–4 follow from the millipede theorem of Pycia and Troyan (2023b), as explained in footnote 36. Thus, we focus on establishing property 5.

Given a subset of objects $X' \subseteq \mathcal{X}$ and a preference ranking for agent i , $>_i$, let $Top(>_i, X')$ be the highest $>_i$ -ranked object in the set X' . Following Bade and Gonczarowski (2017) (hereafter BG), given some history h , we call the maximal superhistory of the form $h' = (h, a^*, \dots, a^*)$ a **terminating history** and the agent who moves at h' a **terminator**; when it is clear what the terminating history is, we denote this agent by t . The terminating history provides an upper bound on the number of passes that can be taken in a row: at the terminating history, the terminator has only clinching actions. Note that the definition of the terminating history is only a function of the game form Γ , and is independent of the players' strategies; it also does not invoke the concept of a lurker.

Our proof of Lemma 2 builds on Lemmas 11–18 below. These lemmas are stated relative to the initial passing path of the game to the terminating history of this path; that is, for all h in these lemmas, we assume that $h = (h_\emptyset, a^*, \dots, a^*)$ for some (possibly zero) number of passing actions a^* .⁵⁴ This will be sufficient because we recursively show that we can restrict attention to Pareto efficient millipede games with a recursive “rounds” structure and these lemmas will be directly applicable to the continuation game at the beginning of each round.

The results below combine the structure of millipede games with the properties of general Pareto efficient obviously strategy-proof games identified by BG. For completeness, we provide proofs of all lemmas. Some of these lemmas follow immediately from BG, in which case the proof is just a reference to the corresponding BG lemma. Some others have a BG “analogue” or can be inferred from BG, but require combining different parts of their analysis with additional arguments to apply to our millipede structure and lurker definition. When there is a BG analogue, we note it in the statement of the lemma. Any proofs not found here can be found in the Supplemental Material (Pycia and Troyan (2026)).

⁵⁴Recall that there is at most one passing action at any history, so this is uniquely defined. If the first agent i takes a clinching action at $h_\emptyset$, and clinches object x , then at the history $h = (h_\emptyset, a_x)$, no other agent has yet moved, and so h is the beginning of a smaller Pareto efficient millipede game on the agents $\mathcal{N} \setminus \{i\}$ and objects $\mathcal{X} \setminus \{x\}$, and we just make the same arguments starting from h . Notice also that if there are lurkers at a history h , the path from $h_\emptyset$ to h must contain a passing action.

Recall that $\mathcal{L}(h)$ and $\mathcal{X}^{\mathcal{L}}(h)$ are the sets of lurkers and lurked objects, respectively, at history h . Let $\mathcal{L}^{\text{BG}}(h)$ and $\mathcal{X}^{\mathcal{L}, \text{BG}}(h)$ denote the sets of BG lurkers and BG-lurked objects. By definition, at any h , any lurker is also a BG lurker: $\mathcal{L}(h) \subseteq \mathcal{L}^{\text{BG}}(h)$.

We start with two lemmas that show that once an agent becomes a lurker, she is never called to move again along the passing path (she might be called to move again, but only after some other agent has clinched an object), and that once they are a lurker, they do not “stop” being a lurker along the passing path.

LEMMA 6: *Let $h^0 = h_\emptyset$ and $h^n = (h_\emptyset, a^*, \dots, a^*)$ be the history reached by taking n passing actions from $h_\emptyset$. Let h^N be the terminating history. Take some h^n and $\ell \in \mathcal{L}^{\text{BG}}(h^n)$. Then, ℓ does not move at $h^{n'}$ for any $n' \in \{n, n+1, \dots, N\}$. In particular, the same is true for lurkers: if $\ell \in \mathcal{L}(h^n)$, then ℓ does not move at $h^{n'}$ for any $n' \in \{n, n+1, \dots, N\}$.*

LEMMA 7: *Let $h^0 = h_\emptyset$ and $h^n = (h_\emptyset, a^*, \dots, a^*)$ be the history reached by taking n passing actions from $h_\emptyset$. Let h^N be the terminating history. For all $n = 1, \dots, N$, $\mathcal{L}^{\text{BG}}(h^{n-1}) \subseteq \mathcal{L}^{\text{BG}}(h^n)$. Further, the same is true for lurkers: $\mathcal{L}(h^{n-1}) \subseteq \mathcal{L}(h^n)$.*

The next lemma states that if a BG-lurked object is previously clinchable for an active BG non-lurker, then h is a terminating history, and this BG non-lurker is the terminator.

LEMMA 8—BG Lemma E.14: *Let $h = (h_\emptyset, a^*, \dots, a^*)$ be a history such that there is an active BG non-lurker j such that $x \in C_j^{\subseteq}(h)$ for some object x that is BG-lurked at h . Then, h is a terminating history, and j is the terminator.*

The next two lemmas relate lurkers to BG lurkers. They show that there can be at most one BG lurker that is not a lurker; further, if such an agent exists, it is the youngest BG lurker, and we have reached the terminating history.

LEMMA 9: *Let $h^0 = h_\emptyset$ and $h^n = (h_\emptyset, a^*, \dots, a^*)$ be the history reached by taking n passing actions from $h_\emptyset$. Let h^N be the terminating history. For all $n < N$, $\mathcal{L}(h^n) = \mathcal{L}^{\text{BG}}(h^n)$.*

LEMMA 10: *At any $h = (h_\emptyset, a^*, \dots, a^*)$, there is at most one BG lurker that is not a lurker. Further, if such an agent i exists, then: (i) i is the youngest BG lurker, (ii) i moves at the immediate predecessor of h , and first becomes a BG lurker at h , (iii) h is a terminating history, (iv) the set of active agents at h is $A(h) = \mathcal{L}(h) \cup \{i, t\} = \mathcal{L}^{\text{BG}}(h) \cup \{t\}$, where the agent t is the terminator and t moves at h , and (v) t is not a lurker at h .*

The next two lemmas show that each lurker lurks a unique object and that there are at most two non-lurkers at any history.

LEMMA 11—BG Lemma E.16: *Let $\mathcal{L}(h) = \{\ell_1^h, \dots, \ell_{\lambda(h)}^h\}$ be the set of lurkers at $h = (h_\emptyset, a^*, \dots, a^*)$ and $\mathcal{X}^{\mathcal{L}}(h) = \{x_1^h, \dots, x_{\lambda(h)}^h\}$, with ℓ_1^h lurking x_1^h , ℓ_2^h lurking x_2^h , etc., where $m < m'$ if and only if ℓ_m^h became a lurker at a strict subhistory of the history at which $\ell_{m'}^h$ became a lurker. Then:*

1. $x_1^h, \dots, x_{\lambda(h)}^h$ are all distinct objects.
2. For all $m = 1, \dots, \lambda(h)$, $P_{\ell_m^h}(h) = \mathcal{X}(h) \setminus \{x_1^h, \dots, x_{m-1}^h\}$.

PROOF: Because any lurker is a BG lurker, and the same applies to lurked objects, this is immediate from BG Lemma E.16 and our Lemma 10, which shows that only the youngest BG lurker might fail to be a lurker. Q.E.D.

LEMMA 12—BG Lemma E.19: *For all $h = (h_\emptyset, a^*, \dots, a^*)$, $|\mathcal{A}(h) \setminus \mathcal{L}(h)| \leq 2$.*

PROOF: If $\mathcal{L}(h) = \mathcal{L}^{\text{BG}}(h)$, then this follows from BG Lemma E.19, which shows there can be at most two active BG non-lurkers at any h . If $\mathcal{L}(h) \neq \mathcal{L}^{\text{BG}}(h)$, by Lemma 10, h is a terminating history and the active agents are $\mathcal{A}(h) = \mathcal{L}(h) \cup \{i, t\}$, where t is the terminator, and neither i nor t are lurkers. Thus, there are at most two non-lurkers at h . Q.E.D.

The next lemma shows that at a history h , all unlurked objects have been previously offered to every lurker, while for any non-lurker, such objects have been previously offered or are still possible.

LEMMA 13—BG Lemma E.11: *If agent i is active at a history h , then $\bar{\mathcal{X}}^\mathcal{L}(h) \subseteq P_i(h) \cup C_i^\subseteq(h)$. If $i \in \mathcal{L}(h)$, then $\bar{\mathcal{X}}^\mathcal{L}(h) \subseteq C_i^\subseteq(h)$.*

Lemma 13 has the following corollary, which will be useful in the proofs of Lemmas 3, 4, and 5 later.

COROLLARY 1: *If, at history h , agent i of type $\succ_i$ clinches $x \in \bar{\mathcal{X}}^\mathcal{L}(h)$ that is unlurked at h , then $x = \text{Top}(\succ_i, \bar{\mathcal{X}}^\mathcal{L}(h))$.*

PROOF: By Lemma 13, all unlurked objects have either been clinchable at some sub-history of h , or are still possible. Thus, if $x \neq \text{Top}(\succ_i, \bar{\mathcal{X}}^\mathcal{L}(h))$, it would not be obviously dominant for agent i to clinch $x \in \bar{\mathcal{X}}^\mathcal{L}(h)$ at h , a contradiction. Q.E.D.

The next lemma is an analogue of Lemma 8 for non-lurkers (who may not be BG non-lurkers).

LEMMA 14: *Let $h = (h_\emptyset, a^*, \dots, a^*)$. If i is an active non-lurker at h and $x_\ell \in C_i^\subseteq(h)$ for some lurked object $x_\ell \in \mathcal{X}^\mathcal{L}(h)$, then $i_h = i$, $P_i(h) = G_i(h) = C_i(h)$, and there is no passing action at h (that is, h is a terminating history).*

The next lemma collects some results about the terminator that will be useful in the proofs of Lemmas 3, 4, and 5 later (parts (i)–(iii) of this lemma follow readily from BG); we show the following:

LEMMA 15—BG Lemmas E.18 and E.20: *Let $h = (h_\emptyset, a^*, \dots, a^*)$ be a history with lurked objects. Let $i_{h'} = t$ be the agent who moves at the terminating history $h' = (h, a^*, \dots, a^*)$. Then:*

- (i) *Agent t is not a lurker at h .*
- (ii) *$C_t^\subseteq(h') = \mathcal{X}$.*
- (iii) *If $i_h \neq t$, then $C_{i_h}(h) \cap C_t^\subseteq(h) = \emptyset$.*
- (iv) *If $x_\ell \in P_j(h)$ for some non-lurker j and lurked object $x_\ell \in \mathcal{X}^\mathcal{L}(h)$, then $j = t$.*

The next lemma shows how, the first time an object is clinched after a series of passes, to determine the assignments of the active lurkers (if any).

LEMMA 16—BG Lemma E.17: Let $h = (h_\emptyset, a^*, \dots, a^*)$, and let i be the agent who moves at h . Let $a_x \in A(h)$ be a clinching action that clinches x for i . Let $\mathcal{L}(h) = \{\ell_1^h, \dots, \ell_{\lambda(h)}^h\}$ be the set of lurkers at h and $\mathcal{X}^{\mathcal{L}}(h) = \{x_1^h, \dots, x_{\lambda(h)}^h\}$ the set of lurked objects, with ℓ_1^h lurking x_1^h , ℓ_2^h lurking x_2^h , etc., where $m < m'$ if and only if ℓ_m^h is an older lurker than $\ell_{m'}^h$ (these sets may be empty). Consider a type profile $\succ_N$ that reaches (h, a_x) . Let $\bar{h} \supseteq (h, a_x)$ be the unique terminal history that is reached by following strategy profile $S(\succ_N)$ starting from (h, a_x) . The object assigned to each agent in $\mathcal{L}(h)$ at $\bar{h}$ can be determined using the following procedure:

- If $x \in \mathcal{X}^{\mathcal{L}}(h)$, each lurker ℓ_m^h receives her lurked object, x_m^h .
- If $x = x_{m_1}^h$ for some lurked $x_{m_1}^h$, then all older lurkers $\ell_{m'}^h$ for $m' < m_1$ receive their lurked object. Lurker $\ell_{m_1}^h$ receives $x' = \text{Top}(\succ_{\ell_{m_1}^h}, \mathcal{X} \setminus \{x_1^h, \dots, x_{m_1}^h\})$.
 - If $x' \in \mathcal{X}^{\mathcal{L}}(h)$, all remaining lurkers $\{\ell_{m_1+1}^h, \dots, \ell_{\lambda(h)}^h\}$ receive their lurked objects.
 - If $x' = x_{m_2}^h$ for some lurked object $m_2 > m_1$, all older unmatched lurkers (ℓ_m^h for $m_1 < m' < m_2$) receive their lurked objects. Lurker $\ell_{m_2}^h$ receives $x'' = \text{Top}(\succ_{\ell_{m_2}^h}, \mathcal{X} \setminus \{x_1^h, \dots, x_{m_2}^h\})$.

This procedure continues with each lurker whose lurked object is taken by an older lurker receiving her top remaining object among all that have not been assigned, until some agent (either i , or some lurker in $\mathcal{L}(h)$) receives an object that was unlurked at h , denoted $\hat{x}$. All remaining unmatched lurkers receive their lurked objects.

PROOF: By Lemma 6, the agent i who moves at h is not a BG lurker, and so we can apply BG Lemma E.17. As every lurker is a BG lurker, the result follows by BG Lemma Q.E.D.

Lemma 16 allows us to calculate the objects received by all active lurkers after the first clinching action following a sequence of passes (as well as the agent i who takes this first clinching action at h and who, as noticed in the proof, is not a lurker at h). Exactly one of these agents receives an object that is unlurked at h , and we let $\hat{x}$ denote this unlurked object. If the object x clinched by the agent i who moves at h is unlurked, then $\hat{x} = x$; otherwise, $\hat{x}$ is assigned to one of the lurkers in $\mathcal{L}(h)$.

The assignments from Lemma 16 can be implemented as follows. After i clinches x at h , then all lurkers at h (if any exist) who lurked an object older than x have immediately clinched the object they lurked at (h, a_x) (if i clinches an unlurked object, i.e., $x = \hat{x}$, then this is all lurkers at h). These lurkers may but do not need to be asked to move following h . If they are called to move, by properties 3 and 4 of Pareto efficient millipedes, they have only clinching actions for their lurked object (their action does not affect their own object, but may influence the continuation game), and they are never called to move again. If x is a lurked object, then the agent who lurked x at h , say ℓ , is the oldest lurker who does not receive their lurked object. Following the move by i at h and possible moves by the older lurkers who clinched their lurked objects at (h, a_x) , by properties 3 and 4 of Pareto efficient millipedes, ℓ is called to move and is offered for clinching all objects that are possible for them, which consist of all lurked objects younger than the one they lurked at h , plus all unlurked objects at h ; they do not have passing actions.⁵⁵ If ℓ takes a lurked object, say x' , then we repeat the above process whereby lurkers who lurked objects older

⁵⁵It is possible that some lurker ℓ , for whom there is only one object in $P_\ell(h)$ minus the set of objects taken by older lurkers, is not asked to move (this case can only happen for the youngest lurker at the end of the game).

than x' but younger than x may or may not be called to move (and if they do move, they have only clinching actions for their lurked object), then the lurker who lurked x' is called to move, etc., until someone (either i , the agent who moved at h , or one of the lurkers at h) clinches an unlurked object, $\hat{x}$. After this clinching of $\hat{x}$, all remaining lurkers have clinched their lurked object. As above, these agents may or may not be called to move, and if they are called to move, they have only clinching actions, and are never called to move again. At any of the above histories, we only include moves that some type of the moving agent takes, thus ensuring that this part of the game is pruned.

REMARK 4: The above paragraph describes how lurkers (if any exist) are called to move after an initial clinching action is taken at a history h (though as described, not all need to be called to move). We use the notation $\hat{h}$ to denote the history that is reached after the completion of all such moves by lurkers. That is, $\hat{h}$ is the smallest history such that $\hat{h} \supseteq (h, a_x)$ and no lurker at h is called to play at any continuation history $h' \supseteq \hat{h}$ (if no lurkers at h are called to play following h —including the case that none exist—then $\hat{h} = (h, a_x)$). How the game proceeds starting from $\hat{h}$ is the subject of the next lemmas.

As there can be at most two non-lurkers at h (Lemma 12), and the agent who moves (and clinches) at h is not a lurker at h (Lemma 6), there might be at most one active agent at h whose assignment is not determined by Lemma 16. As we show next, if such an agent j exists, then it will be this agent who moves at $\hat{h}$. The assignment of this agent (that is, j) is the subject of the next two lemmas. Lemma 17 considers the case where $\hat{x} \in C_j^{\subseteq}(h)$, while Lemma 18 considers the case where $\hat{x} \notin C_j^{\subseteq}(h)$. Note that in these lemmas, j is a non-lurker, but may or may not be a BG lurker.

LEMMA 17: *Consider the environment described in Lemma 16 and assume there exists an active non-lurker $j \in \mathcal{A}(h) \setminus \mathcal{L}(h)$ who does not move at h (thus $j \neq i$) and is such that $\hat{x} \in C_j^{\subseteq}(h)$, where $\hat{x} \in \bar{\mathcal{X}}^{\mathcal{L}}(h)$ is the unlurked object from Lemma 16. Then, at $\hat{h}$, j receives $\text{Top}(>_j, \bar{\mathcal{X}}^{\mathcal{L}}(h) \setminus \{\hat{x}\})$.*

If an agent j described as in Lemma 17 exists, then by properties 3 and 4, agent j is called to move at the history $\hat{h}$ defined in Remark 4. At this move, agent j is offered at least one clinching action for each object that remains possible for her. There is no passing action.⁵⁶

The final case to consider is when there is an active non-lurker j' at h who does not move at h , and $\hat{x} \notin C_{j'}^{\subseteq}(h)$.

LEMMA 18: *Consider the environment described in Lemma 16 and assume there exists an active non-lurker $j \in \mathcal{A}(h) \setminus \mathcal{L}(h)$ who does not move at h (thus $j \neq i$) and is such that $\hat{x} \notin C_j^{\subseteq}(h)$, where $\hat{x} \in \bar{\mathcal{X}}^{\mathcal{L}}(h)$ is the unlurked object from Lemma 16. Let $\hat{h} \supseteq (h, a_x)$ be the history defined in Remark 4. Then, there exists an equivalent millepede mechanism that satisfies properties 1–5 and in which j moves at $\hat{h}$ and $C_j^{\subseteq}(h) \subseteq C_j^{\subseteq}(\hat{h})$ (there may be additional clinching options and/or a passing action as well).*

⁵⁶Also, note that while the lemma says that j receives her top object from $\bar{\mathcal{X}}^{\mathcal{L}}(h) \setminus \{\hat{x}\}$, pruning implies that when j moves, she need not be actually offered all of these objects at her move; in particular, the mechanism only offers her those objects which were possible for her when she entered the game and have not yet been clinchable for her.

The above lemmas establish property 5 of Pareto efficient millipede games for the first round described in Remark 2. To extend these properties to the subsequent rounds, and hence the entire game tree, we define the initiating history of the next round, $h_\emptyset^2$, as follows. Let $h_\emptyset^2 = \hat{h}$, except if there exists a j as in Lemma 17 (in particular, $\hat{x} \in C_j^\subseteq(h)$); in the latter case, $h_\emptyset^2 = (\hat{h}, a_j)$, where a_j is the clinching action taken by agent j who moves at $\hat{h}$. In the former case (i.e., $h_\emptyset^2 = \hat{h}$), if there is a j as in Lemma 18 (in particular, $\hat{x} \notin C_j^\subseteq(h)$), agent j moves at $h_\emptyset^2$, by Lemma 18. If no such j exists, then some previously inactive agent moves at $h_\emptyset^2$ (unless all agents have clinched an object, in which case the game ends).

The continuation game starting at the new initiating history $h_\emptyset^2$ is another Pareto efficient and pruned millipede game on the remaining unmatched agents and objects.⁵⁷ The characterization implied by the above lemmas continues to apply (where the new initiating history $h_\emptyset^2$ plays the role of $h_\emptyset$), and so the continuation game starting from $h_\emptyset^2$ has the same passing-and-clinching structure. We repeat these arguments throughout the game tree by applying the above lemmas starting from $h_\emptyset^k$, the initiating history of round k . This completes the proof of Lemma 2.

C.2. Proofs of Lemmas 3, 4, and 5

In the proofs that follow, we refer to roles in a game form Γ to state properties of Γ that are independent of the specific agent that is assigned to that role. Analogously to the sets of clinchable and possible objects for agents in a game, we write $C_r(h)$ to refer to the set of outcomes that are clinchable for the role $r \in \mathcal{R}$ at h and $P_r(h)$ for the set of outcomes that are possible for role r . Note that these sets do not depend on the role assignment function σ , and if for a particular role assignment, $\sigma(r) = i$, then $C_i(h) = C_r(h)$, $P_i(h) = P_r(h)$, etc. Analogously to the sets $\mathcal{A}(h)$ and $\mathcal{L}(h)$ for active agents and lurkers at a history h , we write $\mathcal{A}_R(h)$ for the set of active roles at a history h , and $\mathcal{L}_R(h)$ for the set of roles that are lurkers at h . When we want to refer to the game form with agents assigned to roles via a specific role assignment function σ_A , we write Γ_A .⁵⁸ In the proofs, we sometimes move fluidly between agents and roles; to avoid confusion, we use the notation i, j, k to refer to specific agents, and the notation r, s, t to refer to generic roles. Finally, note that while the set of agents who are lurkers at any h may differ depending on the role assignment function, the set of lurked objects, the order in which they become lurked, and the set of lurker roles depend only on the game form, and are independent of how the specific agents are assigned to the roles in the game form.

Unless otherwise specified, when we write the phrase “ i clinches x at h ” (or similar variants), what is meant is that i moves at h , takes some clinching action $a_x \in \mathcal{A}(h)$, and receives object x at all terminal histories $\bar{h} \supseteq (h, a_x)$.

The following is a direct consequence of part (iv) of the definition of a lurker, but deserves an emphasis, as it arises frequently in the arguments below.

REMARK 5: If, at a history $h = (h_\emptyset^k, a^*, \dots, a^*)$ in a passing phase of the game (where $h_\emptyset^k$ is the initiating history of the round k that contains h), $x \in C_j(h)$ for an active

⁵⁷In particular, when restricted to the objects that are not yet clinched at $h_\emptyset^2$, all possible ranking types are present at $h_\emptyset^2$ for all agents. For those agents who have never been called to move, this is obvious, as they have not yet revealed any preference information. For the agent j who might be carried over from the previous round, it follows because each type that ranks $\hat{x}$ first (over the set of all objects $\mathcal{X}$) reaches $h_\emptyset^2$, and $\hat{x}$ is no longer available at $h_\emptyset^2$.

⁵⁸To avoid layered subscripts, we write Γ_A instead of Γ_{σ_A} .

non-lurker j at h , then x cannot become the next lurked object along the passing path $(h, a^*, \dots, a^*)$.⁵⁹

We first state a preliminary lemma that will be useful in the main proofs of Lemmas 3, 4, and 5.

LEMMA 19: *Consider a round of the mechanism that begins with an initiating history $h_\emptyset^k$, and let i and j be active non-lurkers at a history $h = (h_\emptyset^k, a^*, \dots, a^*)$. Let $y \in \bar{\mathcal{X}}^\mathcal{L}(h)$ be an unlurked object at h . Further, assume that $i_h = i$ and $y \in C_i(h) \cap C_j^\mathcal{C}(h)$. Consider a type $\succ_j$ that reaches h , and define $\bar{x} = \text{Top}(\succ_j, \bar{\mathcal{X}}^\mathcal{L}(h))$. Then, $\bar{x} \succ_j y$.*

Proof of Lemma 3

We show the first statement; the second statement is then an immediate corollary. Suppose agent i is ordered in step k of the ordering algorithm. First consider the case $k = 1$ and let agent i^* be the first agent to clinch in game Γ_σ and let h^* be the history at which i^* clinches; this clinching induces the ordering of the first segment of agents in step 1 of the ordering algorithm. Let $\mathcal{X}^\mathcal{L}(h^*) = \{x_1, \dots, x_n\}$ be the set of lurked objects at h^* ; this set may be empty.

Case: $\mathcal{A}(h^*) = \mathcal{L}(h^*) \cup \{i^*\}$. If i^* clinches an unlurked object $y \in \bar{\mathcal{X}}^\mathcal{L}(h^*)$, then, in Γ_σ , all lurkers get their lurked objects (the oldest lurker ℓ_1 gets x_1 , the second oldest lurker ℓ_2 gets x_2 , etc.), and in the resulting SD f_σ , the agents are ordered $f_\sigma : \ell_1, \ell_2, \dots, \ell_n, i^*$. By Lemma 11, for each lurker ℓ_m , we have $P_{\ell_m}^h(h) = \mathcal{X}(h) \setminus \{x_1^h, \dots, x_{m-1}^h\}$ and hence $x_m = \text{Top}(\succ_{\ell_m}, \mathcal{X} \setminus \{x_1, \dots, x_{m-1}\})$. When it is agent ℓ_m 's turn in the SD, she is offered to choose from $\mathcal{X} \setminus \{x_1, \dots, x_{m-1}\}$, and thus selects x_m . Finally, consider agent i^* . In game Γ_σ , when she clinches y at h^* , it is unlurked. By Corollary 1, $y = \text{Top}(\succ_{i^*}, \bar{\mathcal{X}}^\mathcal{L}(h^*))$. At her turn in the SD, the set of objects remaining is precisely $\bar{\mathcal{X}}^\mathcal{L}(h^*)$, and so i^* selects y .

In the remaining possibility, i^* clinches some lurked object x_m . Then all older lurkers $\ell_1, \dots, \ell_{m-1}$ get their lurked objects in Γ_σ , and the resulting SD begins as $f_\sigma : \ell_1, \dots, \ell_{m-1}, i^*$. By an argument equivalent to the previous paragraph, each of the lurkers once again gets the same object under the SD. For agent i^* , since she took a lurked object x_m at h^* in Γ_σ , we have $x_m = \text{Top}(\succ_{i^*}, \mathcal{X})$.⁶⁰ Thus, at her turn in the SD, she selects x_m , since it is still available. Then, in Γ_σ , agent ℓ_m is offered to clinch anything from $\mathcal{X} \setminus \{x_1, \dots, x_m\}$. If ℓ_m takes another lurked object $x_{m'}$ for some $m' > m$, then each lurker $\ell_{m+1}, \dots, \ell_{m'-1}$ is assigned to their lurked object, and we add to the SD order as

⁵⁹If x is already lurked at h , it cannot become lurked (and further, by Lemma 14, h is a terminating history so there is no further passing path). Thus, suppose x is unlurked and it first becomes lurked at a history $h' = (h, a^*, \dots, a^*)$, and no other object (besides those already lurked at h , if any) becomes newly lurked between h and h' . Then, some agent i moves at the immediate predecessor of h' , say h'' , and becomes a lurker for x at $h' = (h'', a^*)$. Clearly, $i \neq j$ (as j has been offered x previously, she cannot lurk x), and as no other object has become lurked (or clinched) at h'' , j is an active non-lurker at h'' and $x \in C_j^\mathcal{C}(h'')$. Therefore, condition (iv) in the definition of a lurker is violated. An equivalent statement can be obtained by replacing " $C_j(h)$ " with " $C_j^\mathcal{C}(h)$," using the same argument. Of course, x may eventually become lurked, but not before some other object becomes lurked (or is clinched). This is also an important distinction between lurkers/lurked objects and BG lurkers/lurked objects: it is possible that x is the next BG-lurked object, but if this occurs, we immediately reach the terminating history (by Lemma 14), and we do not classify the agent who BG-lurks x as a lurker according to our definition.

⁶⁰By Lemma 14, h^* is the terminating history, and i^* is the terminator. By Lemma 15, $C_{i^*}^\mathcal{C}(h^*) = \mathcal{X}$, that is, i^* has been offered all objects in $\mathcal{X}$ and chooses x_m , and so $x_m = \text{Top}(\succ_{i^*}, \mathcal{X})$.

$f_\sigma : \ell_1, \dots, \ell_{m-1}, i^*, \ell_{m+1}, \dots, \ell_{m'-1}, \ell_m$. By the same argument as above, at their turn in the resulting SD, each agent $\ell_{m+1}, \dots, \ell_{m'-1}, \ell_m$ gets the same object in the SD.⁶¹ This process continues until someone eventually takes an unlurked object, all remaining lurkers are ordered, and step 1 is completed.

Case: $\mathcal{A}(h^*) = \mathcal{L}(h^*) \cup \{i^*, j\}$ for some $j \in \mathcal{A}(h^*) \setminus (\mathcal{L}(h^*) \cup \{i^*\})$. First consider the case that i^* clinches an unlurked object $y \in \tilde{\mathcal{X}}^\mathcal{L}(h^*)$. If $y \notin C_j^\subseteq(h^*)$, then the argument is exactly the same as in Case (1) (note that j is not ordered in step 1 in this case). If $y \in C_j^\subseteq(h^*)$, then the step 1 partial order is $\ell_1 \tilde{\succ}^1 \dots \tilde{\succ}^1 \ell_n \tilde{\succ}^1 \{i^*, j\}$. We must show that any SD run under $f_\sigma : \ell_1, \dots, \ell_n, i^*, j, \dots$ and $f'_\sigma : \ell_1, \dots, \ell_n, j, i^*, \dots$ result in the same outcome as Γ_σ for these agents. For the lurkers, the argument is as above in either case. For i^* and j , in game Γ_σ , by construction, $y \in C_j(h')$ for some $h' \subsetneq h^*$. Let $z = \text{Top}(\succ_j, \tilde{\mathcal{X}}^\mathcal{L}(h^*))$, and note that by Lemma 19, $z \succ_j y$. Since i clinched y at h^* , we have $y \succ_i z$. In the SD, after all lurkers have picked, the set of remaining objects is precisely $\tilde{\mathcal{X}}^\mathcal{L}(h^*)$. Thus, it does not matter whether i^* or j is ordered next in the SD, as there is no conflict between them: in both cases, i^* takes y , and j takes z , and both f_σ and f'_σ give the same allocation as Γ_σ . For the case where i^* begins by clinching some lurked object $x_m \in \mathcal{X}^\mathcal{L}(h^*)$, we consider agent j and the lurker who, in the chain of assignments, eventually takes an unlurked object y ; otherwise, the argument is analogous.

The proof so far has shown that we get the same allocation for all agents ordered in step 1 of the coding algorithm. Notice that step 1 of the coding algorithm corresponds to round 1 of the Pareto efficient millipede game described in the proof of Lemma 2; that is, the agents coded in step 1 are precisely the agents whose assignments are determined in Lemmas 16 and 17, and thus those whose assignments are determined in the “clinging phase” of round 1, as described in the proof of Lemma 2. If at the end of this clinging phase, all agents have been coded, the algorithm ends and the proof is complete. If there exist remaining unmatched agents, we consider the initiating history $h_\emptyset^2$ that starts the continuation subgame that begins round 2. When there exists an agent j' who was active prior to $h_\emptyset^2$ and whose assignment was not determined by Lemmas 16 and 17, this agent is not coded in step 1 of the coding algorithm and the next round begins with j' moving at $h_\emptyset^2$ and being offered to clinch at least each object in $C_{j'}^\subseteq(h_\emptyset^2)$ (and given possibly more actions as well), as described in the proof of Lemma 2. When no such j' exists, some other previously inactive agent k moves at $h_\emptyset^2$. In either case, step 2 of coding of the original game is the same as step 1 of coding of the continuation subgame starting from $h_\emptyset^2$; note that, like the original game, this continuation game is a Pareto efficient millipede in which the unmatched agents are assigned the unmatched objects. Coding step 2 of the overall game is thus equivalent to the initial coding step on this smaller subgame, and the same arguments show that agents in this coding step receive the same objects under the corresponding serial dictatorship. We continue to proceed recursively in this way through steps 3, $\dots$, K to finish the proof. Q.E.D.

Proof of Lemma 4

First consider $k = 1$ and suppose $\succ_A^1$ is equal to the initial part of the ordering $\succ_B$. Define the function $g_A(i) = |j \in \mathcal{N} : j \succ_A i| + 1$, which is the number of agents ranked

⁶¹When it is agent ℓ_m 's turn in the SD, the set of available objects is a subset of the set of objects that were offered to her when she clinched in Γ_σ : $\mathcal{X} \setminus \{x_1, \dots, x_{m'-1}\} \subseteq \mathcal{X} \setminus \{x_1, \dots, x_m\}$. However, $x_{m'}$ belongs to both sets, and so since ℓ_m takes $x_{m'}$ in Γ_σ , she also takes it at her turn in the SD, when her offer set is smaller.

strictly ahead of i under $\succ_A$, plus 1. We add the “+1” so that this number corresponds to i ’s picking order in the resulting serial dictatorship (i.e., if $\succ_A: i, j, k \dots$, then k will pick third in the resulting serial dictatorship, and $g_A(k) = 2 + 1 = 3$). This will hold for all agents except for those who tie under $\succ_A$; if i and i' tie, then $g_A(i) = g_A(i')$. Define g_B similarly.

CLAIM 1: *If $\succ_A^1$ is equal to an initial segment of $\succ_B$, then $h_A^1 = h_B^1$.*

PROOF OF CLAIM 1: Note that both h_A^1 and h_B^1 consist of a, possibly empty, sequence of passing moves, and so one of these histories must be a subset of the other. Towards a contradiction, assume that $h_A^1 \neq h_B^1$.

First, consider the case $h_A^1 \subsetneq h_B^1$. Define i_A to be the agent that clinches at h_A^1 , and x_A to be the object that is clinched. Since there is a passing action at h_A^1 (because $h_A^1 \subsetneq h_B^1$, which implies $h_B^1 = (h_A^1, a^*, \dots, a^*)$), object x_A is unlurked at h_A^1 , by Lemma 14. Since i_A clinches an unlurked object at h_A^1 , we have $x_A = \text{Top}(\succ_{i_A}, \bar{\mathcal{X}}^{\mathcal{L}}(h_A^1))$ by Corollary 1. By construction of the coding algorithm, $g_A(i_A) = \lambda(h_A^1) + 1$, where $\lambda(h_A^1) = |\mathcal{L}_R(h_A^1)|$ is the number of lurkers (and hence also the number of lurked objects) that are present at h_A^1 . Since $\succ_A^1$ is equal to an initial segment of $\succ_B$ and i_A is ordered in step 1 of Γ_A , we have $g_B(i_A) = \lambda(h_A^1) + 1$ as well.⁶²

We claim that $\mathcal{X}^{\mathcal{L}}(h_A^1) = \mathcal{X}^{\mathcal{L}}(h_B^1)$. First, notice that $h_A^1 \subsetneq h_B^1$ implies $\mathcal{L}_R(h_A^1) \subseteq \mathcal{L}_R(h_B^1)$ and $\mathcal{X}^{\mathcal{L}}(h_A^1) \subseteq \mathcal{X}^{\mathcal{L}}(h_B^1)$, which follows because at each history in the millipede at most one object becomes lurked, and once an object is lurked, it remains lurked along the passing path (by Lemmas 11 and 7). If $\mathcal{X}^{\mathcal{L}}(h_B^1) \supsetneq \mathcal{X}^{\mathcal{L}}(h_A^1)$, then the $(\lambda(h_A^1) + 1)$ th lurked object in Γ_B (denoted $x_{\lambda(h_A^1)+1}$) must be x_A because (i) the coding algorithm puts the agent who receives $x_{\lambda(h_A^1)+1}$ as the $(\lambda(h_A^1) + 1)$ th agent, and hence this agent is i_A , and (ii) by Lemma 3, i_A receives the same object under both σ_A and σ_B . But, because $x_A \in C_r(h_A^1)$, where r is the role that moves at h_A^1 and is not a lurker, x_A cannot be the $(\lambda(h_A^1) + 1)$ th lurked object, by Remark 5, which is a contradiction. Therefore, $\mathcal{X}^{\mathcal{L}}(h_A^1) = \mathcal{X}^{\mathcal{L}}(h_B^1)$. This also means that $\mathcal{L}_R(h_A^1) = \mathcal{L}_R(h_B^1)$ and $\lambda(h_A^1) = \lambda(h_B^1)$; for simplicity, define $\lambda^1 := \lambda(h_A^1) = \lambda(h_B^1)$. Since x_A is unlurked at h_A^1 , it is also unlurked at h_B^1 .

Next, notice that some $j \neq i_A$ moves at h_A^1 in Γ_B , because otherwise, i_A would take the same (clinching) action at h_A^1 in Γ_B , which contradicts $h_A^1 \subsetneq h_B^1$. Let $s = \rho(h_A^1)$ be the role that moves at h_A^1 , and so by definition, $\sigma_A(s) = i_A$ and $\sigma_B(s) = j$. At h_B^1 , there are two active non-lurker roles: role s and another role s' . This follows because role s moves at h_A^1 (and thus is not a lurker at h_A^1), and there is a passing action, so another active non-lurker moves at the history (h_A^1, a^*) . Since $\mathcal{L}_R(h_A^1) = \mathcal{L}_R(h_B^1)$, there are no new lurkers at h_B^1 , and there can be at most two active non-lurkers at any history, it must be role s' that moves at (h_A^1, a^*) , and both roles s and s' remain active non-lurkers at h_B^1 . Further, as only non-lurkers are called to move along the passing path, one of these roles must move at h_B^1 .

We claim that i_A must tie with another agent in $\succ_B$. To see this, note that if role s' moves at h_B^1 , then i_A will tie with agent j in $\succ_B$, since $x_A \in C_s^{\mathcal{L}}(h_B^1)$ and $\sigma_B(s) = j$. If role s moves at h_B^1 , then it is j that clinches at h_B^1 in Γ_B . If j clinches an unlurked object at h_B^1 , then $g_B(j) = \lambda^1 + 1$, and so i_A ties with j in $\succ_B$. If j clinches a lurked object, then role s is the

⁶²This is a key point, and its analogue remains true in the alternate case $h_B^1 \subsetneq h_A^1$. There, $g_B(i_B) = \lambda(h_B^1) + 1$, and we infer that also $g_A(i_B) = \lambda(h_B^1) + 1$. This follows because $h_B^1 \subsetneq h_A^1$ implies $\lambda(h_A^1) \geq \lambda(h_B^1)$, and so at least $\lambda(h_B^1) + 1$ agents are coded in step 1 of $\succ_A$. Thus, at least the first $\lambda(h_B^1) + 1$ agents in $\succ_B$ are in the same position in $\succ_A$, which includes agent i_B .

terminator role, by Lemma 14. Therefore, agent i_A was in the terminator role in Γ_A , and, since she clinched x_A first, we have $x_A = \text{Top}(\succ_A, \mathcal{X})$, which follows because all available objects are possible for the agent in the terminator role, by Lemma 15. This implies that i_A cannot be a lurker at h_B^1 in Γ_B , because if she were, she would have been offered to clinch x_A , and since it is her top object, would have clinched it prior to h_B^1 , by greedy strategies. Thus, the only way for agent i_A to be such that $g_B(i_A) = \lambda^1 + 1$ is if she is an active non-lurker that does not move at h_B^1 , which means that she must tie in $\succ_B$ with some agent.

Thus, we have shown that i_A must tie with some agent k in $\succ_B$, that is, $g_B(i_A) = g_B(k) = \lambda^1 + 1$ for some k . Since i_A is coded in step 1 of Γ_A , and $\succ_A^1$ is equal to an initial segment of $\succ_B$, we further have $g_A(i_A) = g_A(k) = g_B(i_A) = g_B(k) = \lambda^1 + 1$; in other words, agent i_A ties with agent k in both $\succ_A$ and $\succ_B$.

Since i_A ties with k in Γ_A , at h_A^1 , we have $x_A \in C_s^\subseteq(h_A^1)$ for the other active non-lurker role s' at h_A^1 . We have seen that $\sigma_B^{-1}(i_A) \neq s$. If $\sigma_B(s') = i_A$, then in Γ_B , i_A passed at some history $h' \subsetneq h_A^1$ at which she was offered to clinch x_A in Γ_B . By Lemma 19, $\text{Top}(\succ_{i_A}, \mathcal{X}^L(h_A^1)) \succ_{i_A} x_A$, which is a contradiction. Since we know that i_A is coded in step 1 of Γ_B , the only other possibility is that in Γ_B , i_A is a lurker for some object z at h_B^1 , which implies that $z \succ_{i_A} x_A$. It also means that the agent that moves at h_B^1 in Γ_B is clinching a lurked object (because if an unlurked object were clinched, then i_A would be assigned to z , a contradiction). This implies that h_B^1 is the terminating history, by Lemma 14, and $\rho(h_B^1)$ is the terminator role. We cannot have $\rho(h_B^1) = s$, because then role s is the terminator role, and i_A is in the terminator role in Γ_A and would not clinch x_A first in Γ_A , a contradiction. Thus, $\rho(h_B^1) = s'$, and s' is the terminator role. Finally, notice that at h_A^1 , role s is offered x_A and $x_A \in C_{s'}^\subseteq(h_A^1)$, which contradicts Lemma 15, part (iii).

The case $h_B^1 \subsetneq h_A^1$ follows an analogous argument; see footnote 62 for the needed adjustments. Q.E.D.

Thus far, we have shown that if $\succ_A^1$ is equal to the initial part of the ordering $\succ_B$, then $h_A^1 = h_B^1$. We next show that the same roles are coded in step 1 of Γ_A and Γ_B , and further that $\sigma_A(r) = \sigma_B(r)$ for all such roles r .

Define $h^1 := h_A^1 = h_B^1$. In both games, the first clinching is taken by the agent in role $\rho(h^1)$, and the set of lurked objects and active lurker-roles are equivalent at the first clinching in both Γ_A and Γ_B . Letting $r_0 = \rho(h^1)$, write

$$\sigma_A(r_0) \rightarrow x_{a_1} \rightarrow \sigma_A(r_{a_1}) \rightarrow x_{a_2} \rightarrow \cdots \rightarrow \sigma_A(r_{a_M}) \rightarrow x_{a_{M+1}} \quad (\text{A})$$

to represent the chain of clinching that is initiated in Γ_A by agent $\sigma_A(r_0)$ at h^1 : agent $\sigma_A(r_0)$ clinches some (possibly lurked) object x_{a_1} , the agent $\sigma_A(r_{a_1})$ who was lurking x_{a_1} clinches lurked object x_{a_2} , etc., until eventually agent $\sigma_A(r_{a_M})$ ends the chain by being the first agent to clinch an unlurked object $x_{a_{M+1}}$. Similarly, for Γ_B , write

$$\sigma_B(r_0) \rightarrow x_{b_1} \rightarrow \sigma_B(r_{b_1}) \rightarrow x_{b_2} \rightarrow \cdots \rightarrow \sigma_B(r_{b_{M'}}) \rightarrow x_{b_{M'+1}}. \quad (\text{B})$$

Note that the agents who begin the chains, $\sigma_A(r_0)$ and $\sigma_B(r_0)$, are not lurkers in their respective games,⁶³ while all of the remaining agents are lurkers. Also, not all of the agents ordered in step 1 need to appear in the corresponding chain; in particular, any lurker who receives their lurked object does not appear, nor does the other active non-lurker, if such

⁶³If there are no lurkers at h^1 , this is obvious; if there are lurkers, it follows from Lemma 6.

an agent exists. If $M = M'$ and $\sigma_A(r_{a_m}) = \sigma_B(r_{b_m})$ for all $m = 0, \dots, M$, then we say (A) and (B) are **equivalent chains**.

CLAIM 2: *Suppose that (A) and (B) are equivalent chains. Then, the same roles are coded in step 1 in Γ_A and Γ_B , and further, for all such roles, $\sigma_A(r) = \sigma_B(r)$.*

PROOF OF CLAIM 2: By construction of the coding algorithm, the set of roles coded during the coding step initiated at h_A^1 consists of (i) all lurker roles at h_A^1 , (ii) the non-lurker role that moves at h_A^1 , and potentially (iii) the active non-lurker role that does not move at h_A^1 ; label this role s . Since $h_A^1 = h_B^1$, (i) and (ii) are the same in Γ_A and Γ_B . For (iii), role s is coded in Γ_A if and only if the first unlurked object in the chain, $x_{a_{M+1}}$, has been offered to role s to clinch prior to h_A^1 . Since the chains are equivalent, this holds in Γ_A if and only if it holds in Γ_B , which establishes the first statement.

To see that $\sigma_A(r) = \sigma_B(r)$ for all roles that are coded in step 1 of Γ_A (and hence also step 1 of Γ_B), note that because (A) and (B) are equivalent, the statement holds for any role that appears in the chain. For roles that do not appear in the chain, if r' is a lurker role that is active at h^1 , the corresponding lurked object x' is assigned to its lurker in both Γ_A and Γ_B , and so $\succ_A^1$ equivalent to the initial part of the ordering $\succ_B$ implies that $\sigma_A(r') = \sigma_B(r')$ for all such roles, by Lemma 3.

It remains to consider the active non-lurker role s that does not move at h^1 . Note that $M = M'$ and $\sigma_A(r_M) = \sigma_B(r_M)$ implies, by Lemma 3, that $x_{a_{M+1}} = x_{b_{M'+1}}$; let $x_{M+1} := x_{a_{M+1}} = x_{b_{M'+1}}$, and recall that x_{M+1} is unlurked. If there is no such active role s , or if $x_{M+1} \notin C_s^\subseteq(h^1)$, then this role is not coded in step 1, and we are done. Thus, assume that s exists, and that $x_{M+1} \in C_s^\subseteq(h^1)$. In this case, the agent assigned to role s is ordered in step 1 in both Γ_A and Γ_B , and by construction, ties with agent $\sigma(r_M) := \sigma_A(r_M) = \sigma_B(r_M)$ in both $\succ_A$ and $\succ_B$. Once again, $\succ_A^1$ equivalent to the initial part of the ordering $\succ_B$ implies that $\sigma_A(s) = \sigma_B(s)$. Q.E.D.

CLAIM 3: *Chains (A) and (B) are equivalent.*

PROOF OF CLAIM 3: We begin by showing that $\sigma_A(r_0) = \sigma_B(r_0)$. Towards a contradiction, assume that $\sigma_A(r_0) \neq \sigma_B(r_0)$, which implies also that $x_{a_1} \neq x_{b_1}$ by Lemma 3. If $M = M' = 0$, then each of the two chains has only one agent, $\sigma_A(r_0)$ or $\sigma_B(r_0)$, and these agents clinch unlurked objects. Define $\sigma_A(r_0) = i$ and $\sigma_B(r_0) = j$, where $i \neq j$, since they are clinching different objects in their respective games. Since $\succ_A^1$ is equal to the initial part of $\succ_B$, and both i and j clinch unlurked objects, this implies that i and j must tie under $\succ_A$ and $\succ_B$. Thus, by construction of the coding algorithm, there must be another non-lurker role $s \neq r_0$ that is active at h^1 , and $\sigma_A(s) = j$ and $\sigma_B(s) = i$, and $x_{a_1}, x_{b_1} \in C_s^\subseteq(h^1)$. Since i clinches an unlurked object x_{a_1} at h^1 in Γ_A , we have $x_{a_1} = \text{Top}(\succ_i, \mathcal{X}^\subseteq(h^1))$, by Corollary 1. Now, consider game Γ_B . Since $\sigma_B(s) = i$ and $x_{a_1} \in C_s^\subseteq(h^1)$, in game Γ_B , there is some history $h' \subsetneq h^1$ such that $x_{a_1} \in C_i(h')$. By Lemma 19, we have $\text{Top}(\succ_i, \mathcal{X}^\subseteq(h^1)) \succ_i x_{a_1}$, which is a contradiction.

Now, consider the case that $M > 0$. This implies that a lurked object, x_{a_1} , is clinched at h^1 in Γ_A , which means that role r_0 is the terminator role by Lemma 14. It also implies that x_{a_1} is agent $\sigma_A(r_0)$'s favorite object (among all objects $\mathcal{X}$). So, in game Γ_B , agent $\sigma_A(r_0)$ must be lurking object x_{a_1} , that is, she is in role r_{a_1} in Γ_B .⁶⁴ Similarly, agent $\sigma_A(r_{a_1})$ —the

⁶⁴Since agent $\sigma_A(r_0)$ clinches x_{a_1} at the terminating history in Γ_A , we know by Lemma 15 that $x_{a_1} = \text{Top}(\succ_{\sigma_A(r_0)}, \mathcal{X})$. By Lemma 3, agent $\sigma_A(r_0)$ continues to receive x_{a_1} in game Γ_B . Since x_{a_1} is lurked, this object

agent who lurks x_{a_1} in Γ_A —receives x_{a_2} , and so in Γ_B , must be the lurker for x_{a_2} .⁶⁵ As we proceed in this way, agent $\sigma_A(r_{a_2})$ must lurk x_{a_3} in Γ_B , etc., until agent $\sigma_A(r_{a_{M-1}})$, who must lurk object x_{a_M} in Γ_B .

Finally, consider agent $\sigma_A(r_{a_M})$, and, for shorthand, define $k := \sigma_A(r_{a_M})$. We claim that k must be in role s —the active non-lurker role at h^1 that does not move at h^1 —in Γ_B , that is, $\sigma_B^{-1}(k) = s$ (and in particular, such a role s must exist at h^1). To see this, note that k is coded in step 1 of the coding algorithm applied to Γ_A , and receives an unlurked object, so $g_A(k) = \lambda^1 + 1$, and therefore, $g_B(k) = \lambda^1 + 1$. Since at least $\lambda^1 + 1$ agents are coded in step 1 of Γ_B (because $h_A^1 = h_B^1$, as shown above), this is only possible if agent k is also coded in step 1 of Γ_B , and thus k must be active at h^1 . Further, because k lurks x_{a_M} in Γ_A , she strictly prefers x_{a_M} to any younger lurked object and also to any unlurked object. Thus, at h^1 in Γ_B , k cannot be a lurker for any lurked object older than x_{a_M} , because otherwise k would have been offered to clinch x_{a_M} , and by obvious dominance, cannot end up with something she strictly disprefers. Next, k cannot lurk x_{a_M} itself in Γ_B , because as we just argued, agent $\sigma_A(r_{a_{M-1}})$ is in this role in Γ_B . Nor can k be a lurker for any lurked object younger than x_{a_M} . This is because k chose the (unlurked) object $x_{a_{M+1}}$ in Γ_A , which implies $x_{a_{M+1}}$ is her top choice among the set of objects that consist of all unlurked objects plus all lurked objects younger than x_{a_M} (as all of these objects were available to her in Γ_A when she was called to move after the object she lurked, x_{a_M} , was taken by another agent). Finally, k cannot be in role r_0 in Γ_B (the terminator role), because at h^1 , all objects have been offered to the terminator, and thus k would choose to clinch her top object among all of $\mathcal{X}$ (which we have shown is an object different from $x_{a_{M+1}}$), which contradicts Lemma 3. As we know that k must be active at h^1 , the only remaining possibility is $\sigma_B^{-1}(k) = s$.

Because k is coded in step 1 of Γ_B and is in role s , it must be the case that k ties in $\succ_B$, and so there must be some other agent j such that $g_B(j) = \lambda(h^1) + 1$, and so $g_A(j) = g_A(k) = g_B(j) = g_B(k) = \lambda(h^1) + 1$. Since $g_A(j) = \lambda(h^1) + 1$, j must be clinching an unlurked object in Γ_A . Since the first person to clinch an unlurked object in Γ_A is k who clinches $x_{a_{M+1}}$, it must be that $\sigma_A^{-1}(j) = s$ and $x_{a_{M+1}} \in C_s^\subseteq(h^1)$. Finally, since $\sigma_B^{-1}(k) = s$, we have $x_{a_{M+1}} \in C_k^\subseteq(h^1)$ in Γ_B , and by Lemma 19, $\text{Top}(\succ_k, \bar{\mathcal{X}}^\mathcal{L}(h^1)) \succ_k x_{a_{M+1}}$. However, since k chose to clinch $x_{a_{M+1}}$ in Γ_A and $x_{a_{M+1}}$ was unlurked, we have $\text{Top}(\succ_k, \bar{\mathcal{X}}^\mathcal{L}(h^1)) = x_{a_{M+1}}$, which is a contradiction.

The case where x_{b_1} is lurked is analogous, and the argument is omitted. We have thus shown that $\sigma_A(r_0) = \sigma_B(r_0)$.

If agent $\sigma_A(r_0)$ clinches an unlurked object, then the proof is complete. If not, then the above arguments can be repeated to show that $\sigma_A(r_{a_1}) = \sigma_B(r_{b_1})$, etc., until an unlurked object is reached. This completes the proof of Claim 3. *Q.E.D.*

Claims 2 and 3 imply the following:

might be received in Γ_B only by an agent in role r_{a_1} , an older lurker, or the terminator. Agent $\sigma_A(r_0)$ cannot be an older lurker in Γ_B , because then she would have been offered x_{a_1} prior to becoming a lurker, and since it is her top possible object, by greedy strategies, she would have clinched it prior to h^1 . Nor can she be the terminator, because, by assumption, $\sigma_B(r_0) \neq \sigma_A(r_0)$. Therefore, she must be in role r_{a_1} in Γ_B .

⁶⁵This is because by definition of a lurker, agent $\sigma_A(r_{a_1})$ strictly prefers x_{a_1} to all younger lurked objects and all unlurked objects; thus, in Γ_B , she cannot be an older lurker, because she would have been offered x_{a_1} , and thus by obvious dominance could not end up with something she strictly disprefers (recall that by Lemma 3, all agents receive the same objects in both games). She cannot be the terminator, because then, since $h_A^1 = h_B^1$ is the terminating history, all objects would have been offered to the terminator, and so the terminator would receive her top choice among all possible objects which is different than x_{a_2} (the object she received in Γ_A), which is again a contradiction to Lemma 3.

CLAIM 4: *The same roles r' are coded in step 1 of the coding algorithm applied to games Γ_A and Γ_B , and for all these roles, $\sigma_A(r') = \sigma_B(r')$.*

To complete the proof, we establish the claim of the lemma for steps $k > 1$ of the coding algorithm by an inductive argument. Suppose that the lemma obtains for steps $1, \dots, k$ of the coding algorithm. At the conclusion of the round k clinching phase initiated at h_A^k (which is the same as h_B^k), we enter a continuation game on the remaining unmatched agents and objects (round $k + 1$) that begins with the initiating history $h_\emptyset^{k+1}$. By the inductive assumption, this initiating history $h_\emptyset^{k+1}$ is the same under both σ_A and σ_B . As argued in Remark 2, this continuation game continues to have the structure of a millipede mechanism satisfying properties 1–5. Let $h_A^{k+1} \supseteq h_\emptyset^{k+1}$ be the first history at which a clinching action is taken following a (possibly empty) sequence of passes in the subgame of Γ_A starting at $h_\emptyset^{k+1}$; define $h_B^{k+1} \supseteq h_\emptyset^{k+1}$ analogously. If now $\succ_A^{k+1}$ equals to an initial segment of $\succ_B^{k+1}$ restricted to the remaining agents still unmatched at the beginning of step $k + 1$, then we can repeat the arguments developed for $k = 1$ above to show that $h_A^{k+1} = h_B^{k+1}$, the same roles are coded in step $k + 1$ under σ_A and σ_B , and $\sigma_A(r') = \sigma_B(r')$ for all roles coded in step $k + 1$. The inductive argument completes the proof. Q.E.D.

Proof of Lemma 5

As noticed in the main text, for $n = 0$ or $m = 0$ the lemma is immediate. Hence, in the rest of the proof we assume that $n, m \geq 1$. For a (fixed) game form Γ , we let Γ_τ denote the specific game under role assignment σ_τ . Note that the set of objects that are lurked at any given history depends only on the game form, and is independent of the specific role assignment. We use the notation h_τ^* for the first history at which an object is clinched in Γ_τ ; that is, $h_\tau^* = (h_\emptyset, a^*, \dots, a^*)$, where $a^*, \dots, a^*$ are the passes taken by the agents until the agent who moves at h_τ^* chooses to clinch at this history. The number of passes will depend on τ . For any agent j , we write x_j to denote the object that is ultimately received by agent j .

It is without loss of generality to assume that for all games Γ_τ in arm α and its dual arm α' , at least $P + 1$ agents are coded in the first step of the coding algorithm that is initiated at h_τ^* . To see this, note that if not, then the agents coded in the first step are $j_1, \dots, j_{p'}$ for some $p' \leq P$. By Lemma 4, we have that in all of the games under consideration in the two arms, all of these agents are in the same roles, and, at the end of the first coding step, we reach the same history in each game to begin the next coding step. The game that starts at this history is also an efficient millipede, and if the lemma holds true for this smaller game, then it holds for the arms α and α' in the original game. Repeating this argument, we conclude that, to prove the lemma, it is sufficient to prove it for dual arms in which all agents up to at least k_1 (for arm α) or i (for arm α') are coded in the first round of the coding of the game.

Thus, for the entirety of this proof (including all sublemmas stated therein), we assume that the agents $j_1, \dots, j_P$ are all coded in the first round of the games we consider. Since ties can only occur at the very end of a coding step, this implies that each coding in an arm begins $j_1 \succ_\tau j_2 \succ_\tau \dots \succ_\tau j_P \succ_\tau \dots$ and that all x_{j_p} for $p = 1, \dots, P$ are lurked objects at h_τ^* . Letting $\check{h} = (h_\emptyset, a^*, \dots, a^*)$ be the earliest history at which the youngest of these objects becomes lurked (hence, all x_{j_p} are lurked at $\check{h}$), we also have $h_\tau^* = (\check{h}, h, a^*, \dots, a^*)$ for some number (possibly zero) of passing actions a^* . It is possible that $P = 0$, in which case there are no agents j_p .

Since agent i ties in $\succ_1$, she receives an object that is unlurked at h_1^* , which means that $x_i = \text{Top}(\succ_i, \bar{\mathcal{X}}^{\mathcal{L}}(h_1^*))$. By the structure of the sequence, this also implies that for $n' \geq 2$, if $x_i \in \bar{\mathcal{X}}^{\mathcal{L}}(h_{n'}^*)$, then $x_i = \text{Top}(\succ_i, \bar{\mathcal{X}}^{\mathcal{L}}(h_{n'}^*))$ because each of the agents $i, j_1, \dots, j_P$ receives the same object under both σ_1 and $\sigma_{n'}$ (by Lemma 3), and from the game Γ_1 we infer that i prefers the object received (x_i) to all objects except possibly the objects assigned to $j_1, \dots, j_P$, and in game $\Gamma_{n'}$ none of these objects belongs to $\bar{\mathcal{X}}^{\mathcal{L}}(h_{n'}^*)$.

We begin with the following Lemmas 20–24, which show that, under certain conditions, either statement (I) or (II) from Lemma 5 will hold. Then, we apply these lemmas to show that all cases are covered, which will prove the result. The proofs of these lemmas can be found in the Supplemental Material (Pycia and Troyan (2026)).

LEMMA 20: *Assume that there exists an arm α as defined in the statement of Lemma 5, and such that $n \geq 2$. Further, assume that along the initial passing path of the game form, the first lurked objects are (in order) $x_{j_1}, \dots, x_{j_P}, x_{k_1}, \dots, x_{k_{n-1}}$.⁶⁶ Then, at h_{n+1}^* in Γ_{n+1} , there is an agent $\ell \neq j_1, \dots, j_P, k_1, \dots, k_n, i$ such that ℓ is an active non-lurker at h_{n+1}^* that does not move at h_{n+1}^* and $x_i \in C_\ell^{\subset}(h_{n+1}^*)$. Further, i must tie with some other agent in $\succ_{n+1}$, and we label this agent k_{n+1} .*

REMARK 6: A supposition in Lemma 20 is that the first lurked objects of the game form are $x_{j_1}, \dots, x_{j_P}, x_{k_1}, \dots, x_{k_{n-1}}$, in this order, where $n \geq 2$. Given our assumption that objects $x_{j_1}, \dots, x_{j_P}$ are all lurked, a sufficient condition for this to hold is the following: there is a game Γ_A such that $j_1 \cdots j_P \succ_A k_1 \succ_A \cdots \succ_A k_{n-1} \succ_A \{i, k_n\} \succ_A \cdots$ and i is coded in the initial step of the coding algorithm. To see this, assume not, and let n' be the smallest n such that $x_{j_1}, \dots, x_{j_P}, x_{k_1}, \dots, x_{k_{n'-1}}$ become lurked, but $x_{k_{n'}}$ is not the next lurked object. This means that at h_A^* (the history of the first clinching in Γ_A), there are at most $\lambda_A^* = P + n' - 1$ lurked objects. Consider agent $k_{n'}$. By construction, $n' < n$, and so $k_{n'}$ does not tie in $\succ_n$. Thus, in the coding step in Γ_A that begins at h_A^* , agent $k_{n'}$ must be the first agent to clinch an unlurked object. This ends the coding step at $k_{n'}$, without a tie, which contradicts that i is coded in this step in game Γ_A .

LEMMA 21: *Assume that there exists an arm α as defined in the statement of Lemma 5, and such that $n \geq 2$. Consider game Γ_n , and let Q be the step of the coding algorithm in which i is coded in game Γ_n . Let h_n^{q*} be the history at which the first clinching action is taken in step $q \in \{1, \dots, Q\}$ of game Γ_n .⁶⁷ If either (i) $Q = 1$, or (ii) $Q \geq 2$ and at h_n^{1*} there is an agent ℓ that is an active nonlurker at h_n^{1*} that does not move at h_n^{1*} , and $x_i \in C_\ell^{\subset}(h_n^{1*})$, then, in $\succ_{n+1}$, agent i must tie with some agent k_{n+1} .*

LEMMA 22: *Assume that there exists an arm α as defined in the statement of Lemma 5, and such that $n \geq 2$. Further, assume that the $(P + 1)$ th lurked object along the initial passing path of the game form is x_{k_1} . Then, in $\succ_{n+1}$, agent i must tie with some agent k_{n+1} .*

⁶⁶We allow for the possibility that $P = 0$, but whether $P = 0$ or $P > 0$, the assumption that $n \geq 2$ implies that along the initial passing path of the game form, at least x_{k_1} becomes lurked, and is the $(P + 1)$ th lurked object. Note that this only says that there is some history $h = (h_q, a^*, \dots, a^*)$ of the game form at which x_{k_1} is lurked. Whether the history h is reached in any particular game Γ_τ depends on the role assignment and the preference profile of the agents.

⁶⁷That is, $h_n^{q*} = (h_n^q, a^*, \dots, a^*)$ and the agent who moves at h_n^{q*} takes a clinching action. Notice that for $q = 1$, we have $h_n^{1*} = h_n^*$ in our earlier notation.

LEMMA 23: Assume that there exists an arm α as defined in the statement of Lemma 5. If there exists another role assignment function σ_0 with coding of the form

$$j_1 \cdots j_P \succ_0 i \succ_0 k_1 \succ_0 \cdots,$$

then in $\succ_{n+1}$ of arm α , i must tie with some agent k_{n+1} .

REMARK 7—Symmetry: Lemmas 20–23 were stated for arm α , and concluded that i must tie in $\succ_{n+1}$. There are also symmetric versions of these lemmas that apply to arm α' and conclude that k_1 must tie in $\succ'_{m+1}$ that have the exact same proof.

LEMMA 24: Assume that there exist dual arms α and α' as defined in the statement of Lemma 5 such that $n, m \geq 2$. Further, assume that along the initial passing path of the game form, the objects $x_{j_1}, \dots, x_{j_P}$ all become lurked, in this order, but neither x_i nor x_{k_1} is the $(P+1)$ th lurked object. Then, one of the following is true:

1. In $\succ_{n+1}$ of arm α , agent i must tie with some agent k_{n+1} .
2. In $\succ'_{m+1}$ of arm α' , agent k_1 must tie with some agent k'_{m+1} .

With Lemmas 20–24 in hand, we complete the proof of Lemma 5 as follows.

- If there exists σ_0 such that $j_1 \cdots j_P \succ_0 i \succ_0 k_1 \succ_0 \cdots$, then we apply Lemma 23 to arm α to conclude that (I) holds.
- If there exists σ'_0 such that $j_1 \cdots j_P \succ'_0 k_1 \succ'_0 i \succ'_0 \cdots$, then we apply the symmetric version of Lemma 23 (see Remark 7) to arm α' to conclude that (II) holds.

If there do not exist such σ_0 or σ'_0 as in the above bullet points:

- If $n = 1$, then arm α has length $n + 1 = 2$ and the final partial ordering of the arm begins $j_1 \cdots j_P \succ_2 k_1 \succ_2 i \cdots$. Since in this case we assume there is no σ'_0 of the form above (in which i is ranked strictly), it must be that i ties in $\succ_2$, and thus (I) holds.
- If $m = 1$, a similar symmetric argument to the previous bullet point can be made for arm α' to show that (II) holds.
- If $n, m \geq 2$:
 - If x_{k_1} is the $(P+1)$ th lurked object along the initial passing path of the game form, then we apply Lemma 22 to arm α conclude that (I) holds.
 - If x_i is the $(P+1)$ th lurked object along the initial passing path of the game form, then we apply the symmetric version of Lemma 22 (see Remark 7) to arm α' to conclude that (II) holds.
 - If neither x_{k_1} nor x_i is the $(P+1)$ th lurked object along the initial passing path of the game form, we apply Lemma 24 to conclude that either (I) or (II) must hold.

The above covers all possibilities,⁶⁸ which completes the proof of Lemma 5. Q.E.D.

APPENDIX D: EXTENSIONS

So far we have focused on the canonical case in which the number of objects and the number of agents are equal: $|\mathcal{X}| = |\mathcal{N}|$. In this section, we argue that our characterization of Random Priority continues to hold for arbitrary cardinalities of these two sets. Allowing $|\mathcal{X}| < |\mathcal{N}|$ requires us to amend the model by allowing agents to not receive an object. We model this by assigning an “outside option” to agents who do not receive an object

Formally, let $\mathcal{N}$ be the set of agents and $\mathcal{X}$ be the set of objects; we allow arbitrary non-empty finite sets $\mathcal{N}$ and $\mathcal{X}$, imposing no assumptions on their relative cardinalities.

⁶⁸Recall that the lemma is immediate if either m or n is equal to 0, and thus we only need consider $n, m \geq 1$.

We use the notation o to denote the outside option. A deterministic allocation is now a function $\mu : \mathcal{N} \rightarrow \mathcal{X} \cup \{o\}$ such that, for each $x \in \mathcal{X}$, $|\mu^{-1}(x)| \leq 1$; that is, for each $x \in \mathcal{X}$, there is at most one agent $i \in \mathcal{N}$ such that $\mu(i) = x$, while the outside option o can be assigned to an arbitrary number of agents. Each agent $i \in \mathcal{N}$ has a strict preference relation, $\succ_i$, over $\mathcal{X} \cup \{o\}$ and prefers any object (i.e., any $x \in \mathcal{X}$) to the outside option o . Note that if $|\mathcal{X}| \geq |\mathcal{N}|$, then any Pareto efficient mechanism assigns an object to each agent and none of the agents receives the outside option; if $|\mathcal{X}| < |\mathcal{N}|$, then any Pareto efficient mechanism assigns the outside option to exactly $|\mathcal{N}| - |\mathcal{X}|$ agents.

Our main result (Theorem 1) extends to this general environment: *An obviously strategy-proof mechanism is symmetric and Pareto efficient if and only if it is equivalent to Random Priority*. In the remainder of this section, we show how to leverage our base proof to derive this more general result.

If $|\mathcal{X}| \geq |\mathcal{N}|$, the presence of outside options does not affect efficient allocations and our base proof applies with no changes needed.⁶⁹ Suppose hence that $|\mathcal{X}| < |\mathcal{N}|$. The proof for this case has the same structure as the base proof. Steps 1 and 2 of the base proof are unchanged; in particular, Pycia and Troyan (2023b) established all results on which we rely in step 2 for a general class of environments that include the environment with outside options we study now. We can hence again restrict attention to perfect information extensive-form millipede mechanisms with greedy strategies. Let (Γ, S) be such a millipede mechanism with game form Γ and a profile S of greedy strategies. The key difference with the base case is that at any non-terminal history h , the agent who moves at h might be offered to clinch o . We call any such clinching action an ***o-action*** and any history when an o -action is offered an ***o-history***.⁷⁰ We also refer to any agent receiving o at all terminal histories $\bar{h} \supseteq h$ as an ***o-agent*** at h . Denote the set of o -histories as $\mathcal{H}_o$.

Steps 3–6 of the base proof are not directly valid; however, the following proposition allows us to rely on results from these steps without the need to re-prove them.⁷¹

PROPOSITION 3: *Suppose (Γ, S) is a pruned obviously strategy-proof mechanism. If agent i has an o -action at some history h , then this agent is never offered to clinch any object $x \in \mathcal{X}$ at any subhistory of h nor at any continuation history of h . Furthermore, i receives o at all terminal histories $\bar{h}$ such that $\bar{h} \supseteq h$.*

The above implies that if i is an o -agent at h , then she receives o at all terminal histories $\bar{h}$ such that $\bar{h} \supseteq h'$, where h' is either a subhistory of h at which i moves or a continuation history of h .⁷² If (Γ, S) is a millipede mechanism, then given our definition of greedy strategies, we can say more: if a path of successive histories from the root of the game

⁶⁹In particular, the results of Pycia and Troyan (2023b) and Bade and Gonczarowski (2017) on which we build were established for $|\mathcal{X}| \geq |\mathcal{N}|$. We do not rely on $|\mathcal{X}| = |\mathcal{N}|$ directly in our proof either; for example, at no point in our base proof do we infer the existence of agents from the base model assumption that there are at least as many agents as objects. The only equality of cardinalities that matters in the argument is the equality between the number of possible role assignments and the number of possible orderings of agents; this equality—a prerequisite for the existence of a bijection between allocations in a symmetrized millipede and Random Priority—obtains whether $|\mathcal{X}| = |\mathcal{N}|$ holds or not.

⁷⁰There might be multiple o -actions at an o -history which can affect the continuation game.

⁷¹One reason we need to make these adjustments is that the results of Bade and Gonczarowski (2017) we draw on in the base analysis were stated for $|\mathcal{X}| \geq |\mathcal{N}|$. See Appendix E.1 of their paper for a discussion of complications that arise in their results when we allow outside options.

⁷²Note that this does not imply i receives o at all terminal histories; indeed, there may be terminal histories $\bar{h}'$ that do not contain h nor any subhistory of h at which i moves. At such terminal histories $\bar{h}'$, agent i might receive an object from $\mathcal{X}$.

to a terminal history includes an o -history at which agent i moves, then this agent moves exactly once through this path of histories.

PROOF: Consider an agent i that has an o -action at some history h . Suppose i was offered to clinch an object x at h or some subhistory $h' \subseteq h$. By obvious strategy-proofness, at any terminal history that follows h' , including all terminal histories that follow h , this agent must receive either x or an object strictly preferred to x . As the agent strictly prefers x over o , in a greedy strategy this agent would never choose an o -action at any continuation history of h' , including at h . In effect, in a pruned mechanism, o -actions are not offered at such h .

Similarly, if there is some continuation history $h'' \supsetneq h$ at which agent i is offered to clinch an object x or if there is a terminal history $\bar{h} \supseteq h$ at which i receives x , then there must be some action $a \in A(h)$ for which x is possible at (h, a) . Thus, any greedy strategy directs all types of agent i to choose an action other than o -action. In effect, again, in a pruned mechanism, o -actions are not offered at such h . Q.E.D.

As we assume each history in a game form has at least two actions, at any o -history there are at least two o -actions. Let us denote by c^o a collection of possible choices o -agents take at o -histories, that is, for any $h \in \mathcal{H}_o$, $c^o(h) \in A(h)$ and $c^o = (c^o(h))_{h \in \mathcal{H}_o}$. Let $\mathcal{C}^o$ be the set of all possible c^o . Fixing $c^o \in \mathcal{C}^o$, at each o -history h we can remove all actions different from $c^o(h)$ and all of their continuation histories, thus turning each such o -history h into a history at which only one action is offered. If h was the root history, we can hence start the game at the next history. Similarly, if some agents who are not o -agents moved before h , we can let the last move of such agent to lead directly to the first continuation history that is not an o -history; note that such a history exists (as terminal histories are not o -histories) and is uniquely determined (as there is a single action left at any o -history). Proceeding in this way, we create a uniquely-determined game form $\Gamma(c^o)$ that includes no o -histories. Let $S(c^o)$ be the projection of S onto histories in $\Gamma(c^o)$ that correspond to histories we did not remove. The pair $\Gamma(c^o)$ and $S(c^o)$ create a c^o -reduced mechanism $(\Gamma(c^o), S(c^o))$. Note that at any terminal history of $\Gamma(c^o)$, all agents (including o -agents) receive the same outcomes as in the corresponding terminal history of Γ .

So far we have assumed that (Γ, S) is a millipede mechanism. As in the base proof, in steps 3–6, we assume without loss of generality that (Γ, S) is a Pareto efficient millipede mechanism. Because (Γ, S) is Pareto efficient, so is the mechanism $(\Gamma(c^o), S(c^o))$ for any fixed c^o (because the corresponding outcomes are the same). Because (Γ, S) is obviously strategy-proof, so is the mechanism $(\Gamma(c^o), S(c^o))$ for any fixed c^o (because the set of outcomes possible at each history in $(\Gamma(c^o), S(c^o))$ is a weak subset of the set of outcomes possible at the corresponding history in (Γ, S) , and so any time an agent is called to move in the reduced mechanism, the worst-case outcome from her called-for action only improves, while the best case from any alternative only disimproves, and thus if the obvious dominance condition held previously, it will continue to hold in the reduced game). While we still have fewer objects than agents, given the recursive structure of our analysis, all of our results—from steps 3–6—are valid for the mechanisms $(\Gamma(c^o), S(c^o))$.⁷³ We use these results to extend our bijection construction from our base case to the present general case.

⁷³Our analysis (as well as analysis of BG on which we build) is carried out recursively, within endogenous rounds, starting at the root of the game. The analysis of each round does not rely on any aspects of the continuation game beyond the current round other than its efficiency and obvious strategy-proofness. Having fewer objects than agents overall makes no difference for the analysis of any of these rounds.

In order to simplify steps 4–6, we assume that in Γ , all o -agents are given at least one move at some $h \subsetneq \bar{h}$ for any terminal history $\bar{h}$. This assumption is without the loss of generality because if some o -agents do not move by some terminal history $\bar{h}$, we can enlarge the game tree by adding o -histories for all such agents right after the immediate predecessor $h' \subsetneq \bar{h}$ of any terminal history $\bar{h}$.⁷⁴ All newly created terminal histories correspond to the same allocation μ as the original $\bar{h}$. Such enlargement of the game has no impact on its outcome, efficiency, or obvious strategy-proofness properties.

We extend step 4—the construction of the coding algorithm—as follows. Given the mechanism (Γ, S) and the fixed preference profile of the agents, the role assignment determines the profile c^o of all possible o -actions. We construct our new coding in two steps:

- First, we apply our base coding algorithm to $(\Gamma(c^o), S(c^o))$ and obtain a coding that partially orders all agents except for o -agents.
- Second, we extend this partial order to an order on all agents by ranking the o -agents strictly below the other agents. We rank the o -agents strictly in the order in which they first moved.⁷⁵

Given this coding construction, the analogue of Lemma 3 (and hence our step 5) is an immediate corollary of our base Lemma 3.⁷⁶

It remains to extend step 6, that is, to break the ties in the coding while preserving the injectivity of the mapping from role assignment to resulting complete ordering of agents. As a preparation to extend step 6, we define the following partition of the set of role assignments Σ : two role assignments σ and σ' are in the same element of the partition if and only if the resulting class of o -agents is the same under σ and σ' and the order in which o agents are called to move (for their first move) is also the same under σ and σ' . This is well defined, as for this part of the base proof we work with a fixed preference profile, and thus for a given role assignment, the game path is uniquely determined.

Fix an element Π of the above partition of role assignments. As before, each role assignment $\sigma \in \Pi$ determines the profile c_σ^o and hence the game $\Gamma(c_\sigma^o)$ and the profile $S(c_\sigma^o)$ of greedy strategies of the players in this game. Importantly, c_σ^o is the same for all $\sigma \in \Pi$. Hence, if we restrict Lemmas 4 and 5 to role assignments in Π , then the claims of these lemmas remain valid.⁷⁷

The extension of Lemma 4 tells us that the mapping from role assignments in Π to codings is injective. The extension of Lemma 5 allows us to break the ties in the coding of $\sigma \in \Pi$ by using the same procedure as in the base proof provided we only check for arms composed of role assignments from Π . In particular, as before, we start with ties closest to the top of the partial ordering and proceed by breaking one tie at a time. As our construction recognizes fewer arms, the arms we recognize allow us to break fewer

⁷⁴At these o -histories, we can, for example, give each of these agents two actions that lead to identical continuation games, at the end of which all o -agents receive the outside option and all other agents receive the object they clinched before the first added o -action. (This is always possible if $|\mathcal{X}| \geq 2$; if $|\mathcal{X}| \in \{0, 1\}$, the result we are proving becomes straightforward).

⁷⁵In a millipede (Γ, S) , in which greedy strategy directs an agent to clinch their best possible payoff the first time a clinching action is offered, each o -agent actually moves once. Our argument, however, does not rely on this feature.

⁷⁶Indeed, Lemma 3 and its proof are restricted to a single role assignment σ and hence, to obtain its analogue, it is enough to analyze $(\Gamma(c^o), S(c^o))$ for c^o generated by σ .

⁷⁷As with Lemma 3, the proofs of Lemmas 4 and 5 analyze only games created by the role assignments listed in the (respective) statements of these lemmas. Thus, the proofs of these lemmas only require us to work with $(\Gamma(c^o), S(c^o))$ for c^o equal to the c_σ^o common to all $\sigma \in \Pi$. A final additional step needed in the proof of the analogue of Lemma 4 is to notice that the path through the game generated by any role assignment, including all $\sigma \in \Pi$, always has exactly $|\mathcal{N}| - |\mathcal{X}|$ o -agents.

ties than we might have been able to break if we could rely on the full set of arms; as in the base construction, we break the remaining ties arbitrarily.⁷⁸ We first break all the ties in codings of role assignments from Π , before proceeding with other elements of the partition.⁷⁹

For each element Π of the partition, our base argument shows that the mapping from Π to orderings we constructed above is an injection. We claim that the union of these injections over all elements of the partition is an injection from the entire set of role assignments to orderings. Indeed, take any two role assignments, σ and σ' . If they belong to the same element of the partition, then we have just argued that they are mapped to different orderings. If they belong to different elements of the partition, then, by construction of the partition, the o -agents for σ and σ' are either different or move in a different order; as these o -agents are listed in order of their moves at the end of the orderings, the two orderings are hence also different.⁸⁰ As the set of all role assignments and the set of all orderings have the same cardinality ($|\mathcal{N}|!$), the injection from the former to the latter is indeed a bijection, and thus Theorem 1 continues to hold in this more general environment.

REFERENCES

- Abdulkadiroğlu, Atila and Tayfun Sönmez (1998), “Random Serial Dictatorship and the Core From Random Endowments in House Allocation Problems.” *Econometrica*, 66, 689–701. [1604,1605,1608,1612]
- Abdulkadiroğlu, Atila and Tayfun Sönmez (2003), “School Choice: A Mechanism Design Approach.” *American Economic Review*, 93, 729–747. [1605]
- Abdulkadiroğlu, Atila, Yeon-Koo Che, and Yosuke Yasuda (2011), “Resolving Conflicting Preferences in School Choice: The “Boston Mechanism” Reconsidered.” *American Economic Review*, 101 (1), 399–410. [1604]
- Ashlagi, Itai and Yannai A. Gonczarowski (2018), “Stable Matching Mechanisms Are not Obviously Strategy-Proof.” *Journal of Economic Theory*, 177, 405–425. [1615]
- Bade, Sophie (2020), “Random Serial Dictatorship: The One and Only.” *Mathematics of Operations Research*, 45 (1), 353–368. [1605]
- Bade, Sophie and Yannai Gonczarowski (2017), “Gibbard–Satterthwaite Success Stories and Obvious Strategyproofness.” ArXiv Working Paper. [arXiv:1610.04873](https://arxiv.org/abs/1610.04873). [1605,1610,1617,1618,1631,1646]
- Basteck, Christian (2024), “An Axiomatization of the Random Priority Rule.” WZB Discussion Paper SP II 2024-201. [1606]
- Basteck, Christian and Lars Ehlers (2024), “On (Constrained) Efficiency of Strategy-Proof Random Assignment.” Working Paper. [1604,1605]
- Bó, Inácio and Rustamdjan Hakimov (2023), “Pick-an-Object Mechanisms.” *Management Science*. [1604]
- Bogomolnaia, Anna and Herve Moulin (2001), “A New Solution to the Random Assignment Problem.” *Journal of Economic Theory*, 100, 295–328. [1604,1609]
- Brandt, Felix, Matthias Greger, and René Romen (2023), “Characterizing Random Serial Dictatorship.” Working Paper. [1604]
- Breitmoser, Yves and Sebastian Schweighofer-Kodritsch (2019), “Obviousness Around the Clock.” WZB Discussion Paper SP II 2019-203. [1604]
- Carroll, Gabriel (2014), “A General Equivalence Theorem for Allocation of Indivisible Objects.” *Journal of Mathematical Economics*, 51, 163–177. [1605,1612,1613]
- Che, Yeon-Koo and Fuhito Kojima (2010), “Asymptotic Equivalence of Random Priority and Probabilistic Serial Mechanisms.” *Econometrica*, 78 (5), 1625–1672. [1604]

⁷⁸While one might be concerned this might break injectivity, we are about to show that it does not.

⁷⁹As there are no interactions between tie breaking across different elements of the partition, the order in which we handle partition elements, or whether we break only some ties in one partition element, before moving to the next one (to eventually come back to the first) does not matter.

⁸⁰This inference relies on the fact that the path through the game generated by any role assignment, including σ and σ' , always has exactly $|\mathcal{N}| - |\mathcal{X}|$ o -agents.

- Erdil, Aytek (2014), “Strategy-Proof Stochastic Assignment.” *Journal of Economic Theory*, 151, 146–162. [1604]
- Featherstone, Clayton R. and Muriel Niederle (2016), “Boston versus Deferred Acceptance in an Interim Setting: An Experimental Investigation.” *Games and Economic Behavior*, 100, 353–375. [1604]
- Gonczarowski, Yannai A., Ori Heffetz, and Clayton Thomas (2023), “Strategyproofness-Exposing Mechanism Descriptions.” Technical report, National Bureau of Economic Research. [1604]
- Knuth, Donald E. (1996), “An Exact Analysis of Stable Allocation.” *Journal of Algorithms*, 20 (2), 431–442. [1605,1612]
- Li, Shengwu (2017), “Obviously Strategy-Proof Mechanisms.” *American Economic Review*, 107 (11), 3257–3287. [1604,1605,1607,1608,1617,1619,1631]
- Liu, Qingmin and Marek Pycia (2011), “Ordinal Efficiency, Fairness, and Incentives in Large Markets.” [1604]
- Mackenzie, Andrew (2020), “A Revelation Principle for Obviously Strategy-Proof Implementation.” *Games and Economic Behavior*, 124, 512–533. [1615]
- Mandal, Pinaki and Souvik Roy (2022), “Obviously Strategy-Proof Implementation of Assignment Rules: A New Characterization.” *International Economic Review*, 63 (1), 261–290. [1605]
- Miralles, Antonio (2008), “School Choice: The Case for the Boston Mechanism.” Unpublished report, Boston University. [1604]
- Pápai, Szilvia (2000), “Strategyproof Assignment by Hierarchical Exchange.” *Econometrica*, 68, 1403–1433. [1605,1613]
- Pathak, Parag A. and Jay Sethuraman (2011), “Lotteries in Student Assignment: An Equivalence Result.” *Theoretical Economics*, 6 (1), 1–17. [1605,1612]
- Pycia, Marek (2019), “Evaluating With Statistics: Which Outcome Measures Differentiate Among Matching Mechanisms?” Working Paper. [1605]
- Pycia, Marek and Peter Troyan (2023a), “Strategy-Proof, Efficient, and Fair Allocation: Beyond Random Priority.” [1604,1605]
- Pycia, Marek and Peter Troyan (2023b), “A Theory of Simplicity in Games and Mechanism Design.” *Econometrica*, 91 (4), 1495–1526. [1603,1605,1607,1609,1613-1615,1617,1619,1631,1646]
- Pycia, Marek and Peter Troyan (2026), “Supplement to ‘The Random Priority Mechanism Is Uniquely Simple, Efficient, and Fair’.” *Econometrica Supplemental Material*, 94, <https://doi.org/10.3982/ECTA23230SUPP>. [1609,1631,1644]
- Pycia, Marek and M. Utku Ünver (2015), “Decomposing Random Mechanisms.” *Journal of Mathematical Economics*, 61, 21–33. [1604]
- Pycia, Marek and M. Utku Ünver (2017), “Incentive Compatible Allocation and Exchange of Discrete Resources.” *Theoretical Economics*, 12, 287–329. [1605]
- Troyan, Peter (2019), “Obviously Strategy-Proof Implementation of Top Trading Cycles.” *International Economic Review*, 60 (3). [1605]
- Zhou, Lin (1990), “On a Conjecture by Gale About One-Sided Matching Problems.” *Journal of Economic Theory*, 52, 123–135. [1604]

Co-editor Parag Pathak handled this manuscript.

Manuscript received 28 June, 2024; final version accepted 5 May, 2026; available online 6 May, 2026.

All authors assume responsibility for all aspects of the paper.